BACHELORARBEIT im Studiengang

Wirtschaftsinformatik – Bachelor of Science (B.Sc.)

aus dem Fach/Modul: Wirtschaftsinformatik

gestellt von: Professor Dr. Peter Fettke

mit dem Thema: Prozessvorhersage mittels Transformer-Netzen am Beispiel der Fischertechnik APS

bearbeitet von

Name: Nicolas Christian Edelmann

Adresse: Reinhold-Zeller-Str. 19, 66386 St. Ingbert

Abgabetermin: .....

Spätester Beurteilungstermin: .....

Eidesstattliche Erklärung

Ich erkläre an Eides statt, dass ich die vorliegende Arbeit selbstständig und ohne die Beteiligung Dritter verfasst habe und keine anderen Quellen und Hilfsmittel als die angegebenen benutzt habe. Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus Veröffentlichungen oder anderweitigen fremden Äußerungen entnommen wurden, sind als solche kenntlich gemacht. Ich versichere des Weiteren, dass die elektronische Version mit der gedruckten Version übereinstimmt.

Insbesondere bestätige ich hiermit, dass ich alle mittels künstlicher Intelligenz betriebenen Software (z. B. ChatGPT) generierten und/oder bearbeiteten Teile der Arbeit unter Angabe des Prompts, des verwendeten Modells, des Datums und der Zeit der Interaktion kenntlich gemacht und als Hilfsmittel angegeben habe. Ich erkläre mich damit einverstanden, dass die Arbeit mit einer Plagiatssoftware überprüft wird.

Mir ist bewusst, dass der Verstoß gegen diese Versicherung zum Nichtbestehen der Prüfung bis hin zum Verlust des Prüfungsanspruchs führen kann.

Ich habe ausschließlich die folgenden Hilfsmittel benutzt:

- GitHub Copilot Inline Suggestions: Integriert in Visual Studio Code, verwendet es künstliche Intelligenz, um Autovervollständigung und Code-Vorschläge während der Implementierung zu erstellen.

St. Ingbert, 31. Juli 2026 _____

Nicolas Christian Edelmann

Contents

List of Figures

1

Introduction

Traditionally, discovering knowledge about business processes required combining estimates and manual data collection. This acquired knowledge would then be used to analyze, redesign and optimize said processes or to make process-external decisions (Di Francescomarino et al., 2026). This type of knowledge discovery is naturally error-prone and tends to be time-consuming.

Nowadays, more and more businesses rely on enterprise systems to store highly detailed process execution information, based on sensor outputs, workflow data and machine logs. By applying various process mining techniques to these datasets, crucial knowledge can be extracted (Aalst, 2016), which can then be used in multiple business process lifecycles (Muehlen & Ho, 2006). During process *design*, analysis of historical data can provide comparison points, during process *monitoring* outliers and bottlenecks can be detected.

A subfield of process monitoring is predictive process monitoring. It is focused on predicting the behavior of process executions under certain conditions. It can then provide information about the business goals, such as expected failure rates or execution time. If a process is structured into multiple events, *next-event prediction* is concerned about the next event's activity or surrounding metadata. Here, machine learning approaches have been on the rise, from using LSTMs (Camargo et al., 2019; Tax et al., 2017) to modified transformer architectures (Bukhsh et al., 2021; Ni et al., 2023).

Predictive process monitoring needs to face certain challenges. In modern environments, process executions are inherently concurrent. As a result, given a running process execution, **the** next event is not clearly defined: Due to the parallel execution multiple actions could be executed at the same time, or in an arbitrary order depending on system load. While a linear log collected by an enterprise system might exist, due to the parallel execution this order should not be fully enforced onto predictions. As an example, if two machines need to produce A and B, and then a third machine combines them, the imaginary prediction trace

$\texttt{Produce}\ B \rightarrow \texttt{Produce}\ A \rightarrow \texttt{Combine}\ A \wedge B$

does not necessarily conflict with

$$\texttt{Produce}\, A \rightarrow \texttt{Produce}\, B \rightarrow \texttt{Combine}\, A \wedge B$$

However, there does exist a partial order of events within a process due to causal relationships between the events. Following the example from before, clearly

$$\texttt{Combine}\, A \wedge B \rightarrow \texttt{Produce}\, A \rightarrow \texttt{Produce}\, B$$

is not feasible, as one cannot combine non-existing products.

In this work, we present an approach to use transformer networks to solve the next event prediction problem, while working around the concurrency uncertainty by modeling it using the Heraklit infrastructure (Fettke & Reisig, 2021).

We evaluate this approach on the Fischertechnik **Agile Production Simulator** (APS)[1], providing the tools for end-to-end prediction. We extract process logs directly from the internal MQTT broker and then fit and train the model based on derived Heraklit steps. The model achieves an outstanding 91.07% mean accuracy on single next-event prediction. Even when simulating noisy MQTT logs by removing events or adding random events into the logs, similar results can still be achieved.

The structure of the thesis can be summarized as follows. Chapter 2 introduces the necessary background on Heraklit, Transformers and MQTT while providing information on the theoretical prediction approach.

Chapter 3 then explains the chosen modeling approach of the Fischertechnik APS case study using Heraklit. Continuing into the technical details, Chapter 4 provides a deeper dive into the details of the applied approach and the Fischertechnik data.

The case study model is lastly evaluated in Chapter 5 in various scenarios, before drawing conclusions and discussing limitations and future work in Chapter 6.

[1] https://www.fischertechnik.de/de-de/industrie-und-hochschulen/technische-dokumente/simulieren/agile-production-simulation

2

Theory

In this chapter, we will present the theoretical and technical background of this work. This includes defining the Heraklit modeling framework used to model the factory, the process prediction problem and finally the transformer architecture applied to solve it, while providing related work.

2.1 Modeling with Heraklit

Heraklit (Fettke & Reisig, 2021) is a process modeling framework designed to thrive in a discrete digital world, providing a formal foundation for interaction-driven process management of digital and cyber-physical systems.

We will not provide an in-depth explanation of Heraklit, but will focus on an overview of the most important points relevant to this work. In general, Heraklit builds upon three main characteristics:

- **Architecture**: Models can be composed and refined, allowing the building of large systems using the *composition calculus*.
- **Dynamics**: Actions are performed using local state, and dynamics between actions using causal relationships.
- **Statics**: Items, data and operations on them are treated as first-class citizens.

The *composition calculus* of *modules* and causal modeling are what mainly power our approach to process prediction. To understand how they formally work, we will first define the Heraklit notions of some of the terms, including **interface** and **module**, **composition of modules** and a **step module**. These definitions are based on definitions found in (Fettke & Reisig, 2021; 2022). Due to the limited scope of the thesis and the limited requirements of Heraklit in our use case, definitions are not necessarily complete and proofs are left out. They can be read upon in (Fettke & Reisig, 2021).

HERAKLIT modules are conceptually modeled using graphs, with inner vertices and outer vertices. These outer vertices contribute to the *interface* of a module and are the external connection points of a module.

Definition 1 *Interface*: A *labeled* and *totally ordered* set is an interface.

Definition 2 *Match*: Let A and B be two interfaces, let $a \in A$ and $b \in B$. Then $\{a, b\}$ is a *match of A and B*, if for some label λ, both a and b are $\lambda-$labeled, and

$$|\{a' < a \wedge a' \text{ is } \lambda - \text{labeled} \mid A\}| = |\{b' < b \wedge b' \text{ is } \lambda - \text{labeled} \mid B\}|,$$

ensuring the number of $\lambda-$labeled gates that are smaller than a in A is equal to the number of $\lambda-$labeled gates that are smaller than b in B. A gate of A that does not belong to a match of A and B is *match-free* with respect to B.

- Let *matches(A, B)* be the set of all matches of A and B.
- Let *matchfree(A, B)* be the set of all elements of A that do not belong to a match of A and B.

Definition 3 *Module*: A module $M = (V, E)$ is a directed graph together with two interfaces $*M \subseteq M$ and $M* \subseteq M$ of nodes of M. The interfaces $*M$ and $M*$ are the *left* and *right* interfaces of M respectively.

The module composition requires composition of graphs along the interfaces. Intuitively, graph composition of two graphs ensures that all graph nodes still exist in the composed graph, just *merging* the nodes at the interface. The new graph thus contains all nodes of both graphs not included in the interface, the free nodes of both interfaces and once the nodes in the interface. One then just needs to reconstruct the edges as before.

Definition 4 *Graph Composition*: Let M and N be two graphs, let $A \subseteq M$ and $B \subseteq N$ be interfaces. Then the *composition of M and N along A and B* is the graph G where:

1. The nodes of G are

   $$(M \setminus A) \cup (N \setminus B) \cup \text{matchfree}(A, B) \cup \text{matchfree}(B, A) \cup \text{match}(A, B)$$

2. For each edge (x, y) of M or N,
   - if x and y are both match free, then (x, y) is an edge of G;
   - if x is match free and $\{y, y'\}$ is a match, then $(x, \{y, y'\})$ is an edge of G;
   - if $\{x, x'\}$ is a match and y is match free, then $(\{x, x'\}, y)$ is an edge of G;
   - if $\{x, x'\}$ and $\{y, y'\}$ are matches, then $(\{x, x'\}, \{y, y'\})$ is an edge of G.

Definition 5 *Module Composition*: Let A and B be two modules. Their composition $A \bullet B$ is defined as follows:

- The graph of $A \bullet B$ is defined as the composition of the graphs (*Def. 4*) of A and B along the interfaces A^* and $^*B$.
- The left interface $^*(A \bullet B)$ of $A \bullet B$ is $^*A \cup \text{matchfree}(^*B, A^*)$. The elements of $^*A$ are ordered before the elements of $\text{matchfree}(^*B, A^*)$.
- The right interface $(A \bullet B)^*$ of $A \bullet B$ is $B^* \cup \text{matchfree}(A^*, {}^*B)$. The elements of B^* are ordered before the elements of $\text{matchfree}(A^*, {}^*B)$.

Module Composition holds two important properties:

- **Associativity**: Let A, B, C be modules. Then $(A \bullet B) \bullet C = A \bullet (B \bullet C)$. This property allows us to ignore the brackets in composition, and merge multiple submodules in arbitrary orders.
- **Commutativity** without shared gates: Let A, B be modules. $A \bullet B = B \bullet A$ iff A and B share no equal interface labels. This will be of high interest when composing modules without causal relationships. Their interface would not share any labels, so the order of their composition also does not matter.

Note how in *Def. 3* there is no notion of any dynamics yet. Heraklit follows the idea of Petri nets to model the dynamics, so we will now refine our definitions to separate the graph nodes into alternating *places* and *transitions*.

Definition 6 *Net Graph*: Let $G = (V, E)$ be a graph, and let P and T be two disjoint sets with elements called *places* and *transitions*, respectively, such that:

1. $P \cup T = V$;
2. For each edge $(x, y) \in E$, it holds that: either $x \in P$ and $y \in T$, or $x \in T$ and $y \in P$.

Then G is a *net graph*, and we write $G = (P, T; E)$

Definition 7 *Net Module*: For a net graph G, a module over G is called a *net module*.

Composition of net modules is defined via the composition of modules and produces a valid net module.

To model the discrete stepwise behavior of processes, we define *step modules*, which only ever include a single *event* and the states it affects. Speaking in terms of net modules, every step module only contains **one transition**.

Definition 8 *Step Module*: Let $M = (P, \{t\}; E)$ be a net module with disjoint interfaces $^*M$ and M^* with $P = {}^*M \cup M^*$ such that for each $p \in P$ holds: $(p, t) \in E$ iff $p \in {}^*M$, and $(t, p) \in E$ iff $p \in M^*$. Then M is a *step module*.

In the following, we will often refer to *step modules* when describing objects in Heraklit context for simplicity.

Definition 9 *Run Module*: Let $R = (P, T; E)$ be a net module. R is a *run module* iff

1. all places and all transitions are labeled,
2. at each place of R at most one edge begins and at most one edge ends,
3. no edge sequence forms a cycle,
4. $^*R$ contains all places where no edge ends,
5. R^* contains all places where no edge begins.

Important properties of run modules are:
- Each step module is also a run module.
- The composition of run modules generates again a run module.
- Each finite run module R can be composed as $R = P_1 \bullet \ldots \bullet P_n$ from step modules $P_1, \ldots, P_n$.

In the following, *run modules* are often referred to as *runs* for simplicity.

This concludes the most necessary basic Heraklit concepts necessary for our approach. While Heraklit offers many possibilities to model data, structures and functions, we will not need them for the simple model of the Fischertechnik APS.

Section 2.2.1 shows some graphical examples of step modules and composition into runs. In Chapter 3 the step modules for the Fischertechnik APS are defined, along with some examples of composition.

2.2 Process Prediction

Modern enterprise systems collect huge amounts of data of process executions in so-called **event logs**. The event log of just one execution of such a process is called a **trace**. Each event in these logs contains at least an *identifier* to distinctly identify the process execution, and *event name* describing the action and some sort of *ordering* to express the sequence of events, mostly timestamps and execution times. Additional metadata, such as involved resources, machines or sensor data, can also be attached to an event.

Process prediction is concerned with predicting **possible outcomes** of ongoing traces. This prediction can be performed online, meaning while the execution of the process happens, such that unwanted outcomes can be prevented by preemptively changing the execution based on the prediction.

Predictions are thus performed on incomplete traces, providing potential outcomes as outputs. The to-be-predicted outcomes are determined by the business problem at hand. They can be broad information about the full process execution

such as expected remaining process execution time or whether the process will fail or succeed in its action, or fine-grained information, such as the next possible event or detecting anomalies within events (Aalst, 2016; Neu et al., 2022).

In this work, we will focus on predicting the next event of a process.

2.2.1 Next-Event Prediction

Given a full execution trace $t = e_1 \to ... \to e_n$ of length n and let $p = e_1 \to ... \to e_i$ with $i < n$ be a finite prefix of t, the next event might seem to simply be e_{i+1}. This notion is certainly correct in a sense that e_{i+1} is **a** next step of p. However, this definition fails to capture the semantics of concurrent systems, where multiple *correct* linearizations and thus orderings are possible.

We build upon the example from the Introduction:

The process needs to start, then two separate machines produce `A` and `B`, and then a third machine combines them. There are two causal relationships given - `Start` must happen first, and the combination of `A` and `B` must clearly happen after both `A` and `B` were produced. Given the prefix trace `Start`, both `Produce` A and `Produce` B are valid options due to the parallelism given. Both traces

`Start` $\to$ `Produce` $A \to$ `Produce` $B \to$ `Combine` $A \wedge B$

and

`Start` $\to$ `Produce` $B \to$ `Produce` $A \to$ `Combine` $A \wedge B$

can be deemed *correct*. Given the first trace is then later extracted from the system, the second one should still not be deemed incorrect - as we do not want to care about the order of causally unrelated events. Importantly though,

`Start` $\to$ `Combine` $A \wedge B \to$ `Produce` $A \to$ `Produce` B

is **not** a trace we should deem correct.

To define this correctness measure, we use the Heraklit theory. As a first step, we need to create a mapping between events and Heraklit Step Modules. Given this mapping, we can use the following definitions to describe a correct prediction.

Definition 10 *Run Prefix*: Let $R = P \bullet S$ be a run module. Then P is a prefix and S a suffix of R.

Definition 11 *Next Step*: Let R be a run module and P a prefix of R. Then the step module s is a next step after P in R iff $P \bullet s$ is a prefix of R.

Definition 12 *Correct Prediction*: Let R be a run module and P a prefix of R. Then the step module s is a correct prediction of P iff s is a next step after P in R.

Notice how next steps and correct predictions are inherently the same.

We can examine these definitions on our example from above by defining the following steps:

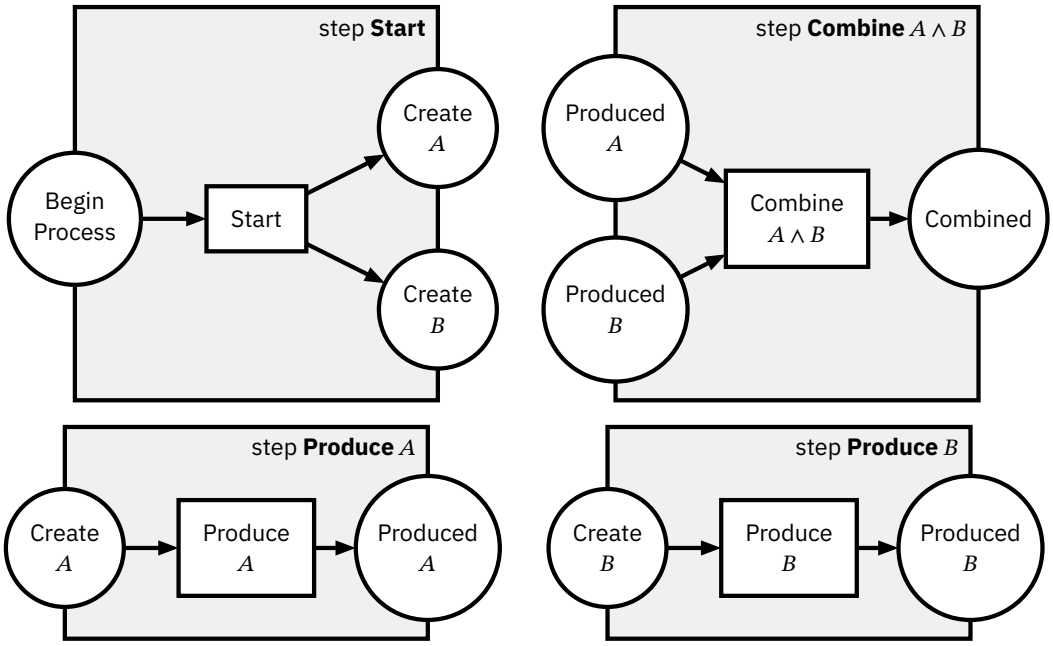


Figure 2.1: *Example Machine Step Modules*

To keep it simple, the labels of the step modules are the same as the labels of the transitions contained within them. The transitions on the left and right on the edge of the module border are the left and right interfaces, respectively. By the layout of the steps, one can already grasp as to how valid runs could look.

By our definitions of composition, we already know that

$$\begin{aligned} &\text{Start} \bullet \text{Produce } A \bullet \text{Produce } B \bullet \text{Combine } A \wedge B \\ =\ &\text{Start} \bullet \text{Produce } B \bullet \text{Produce } A \bullet \text{Combine } A \wedge B \end{aligned}$$

This composition can also be shown graphically, as seen in Figure 2.2. By looking at just the first step, **Start**, one can see how both **Produce** A and **Produce** B are correct predictions for such a run, as no causal relationships are existent between them.

To make use of this definition, we first need to model a system by providing the Heraklit step modules. For our case study, they can be found in Chapter 3.

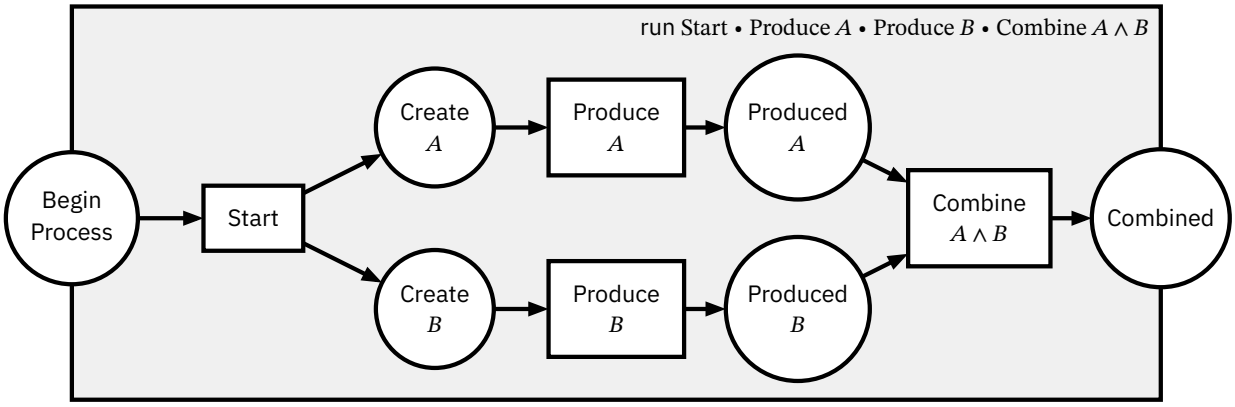


Figure 2.2: *Example Composed Machine Run*

2.2.2 MQTT

MQTT is a client-server protocol using a publish/subscribe pattern. It is considered the most favorable connection protocol for Internet of Things (IoT) applications (Yassein et al., 2017).

With MQTT, clients can connect to the server, a *broker*, and *publish* some information to a *topic*. Other clients can connect to the same broker, and *subscribe* to certain topics. Whenever new messages are published to a topic this client is subscribed to, the broker pushes this message to the client. Clients can be *publishers* (or sources) and *subscribers* (or sinks) at the same time. Thus, MQTT is a many-to-many communication protocol.

It provides three levels of Quality of Service (QoS), which can deal differently with network performance issues like latency or error rate, at the trade-off of network traffic and energy consumption (Toldinas et al., 2019).

1. QoS 0: At most once. The message is sent to all subscribers only once. It is not stored on the broker, and if delivery was not successful, no redistribution is planned. It is also known as *"fire and forget"*.
2. QoS 1: At least once. The message is repeatedly sent to all subscribers that have not acknowledged the message yet, marking every message from the second send on using the `DUP` flag. The message is stored at the broker until all subscribers have received the message.
3. QoS 2: Exactly once. Similar to QoS 1, the broker stores the message and resends it. However, the clients will need to also store the message until it gets released via a special message to ensure no double handling takes place.

The Fischertechnik APS uses MQTT as its main channel of communication between the different production modules. While most state updates are distributed via QoS 1, some specific order requests are executed using QoS 2.

Both cases result in duplicate messages from the broker, which need to be filtered out during data processing. The specific MQTT broker can also *retain* messages of QoS 1, which redistributes the latest message from a topic to newly connected clients, even if that message was published before. These messages need filtering as well, as they can incorrectly influence the assumed state.

We describe the exact *processing* of the Fischertechnik MQTT messages and *conversion* into tokens in Section 4.1.

2.3 Transformer

Over the last decade, research mostly identified deep-learning approaches as an advancement over traditional machine-learning approaches for predictive process monitoring (Di Francescomarino et al., 2026; Evermann et al., 2017; Mehdiyev et al., 2020; Neu et al., 2022). Especially with a lot of different events requiring high cardinality categorical variables, traditional machine-learning approaches start to show their weaknesses (Zhu, 2020).

While using Long Short-Term-Memory models (LSTMs), a special version of recurrent neural networks (RNNs), showed their success (Camargo et al., 2019; Tax et al., 2017), later state-of-the-art models use the transformer architecture (Bukhsh et al., 2021; Ni et al., 2023).

2.3.1 Transformer Architecture

First presented in (Vaswani et al., 2017), this deep-learning based model architecture revolutionized its field, with now more than 250,000 direct citations[2]. Originally intended for language translation, it is now most known for the use in Large Language Models (LLMs), that power platforms like ChatGPT (Kulkarni et al., 2023).

The following section will provide a technical overview of the architecture as presented in the original paper. The transformer can generally be split into two parts: the `Encoder` and the `Decoder`, whereas the former focuses on creating a *contextual understanding* of input data and the latter is responsible for *generating output* sequences based on previous output and the understanding of the `Encoder`. Nowadays often only one of the two structures is used, e.g. BERT only uses an `Encoder` layer to learn text representations (Devlin et al., 2019), while GPT and GPT-2 both used a `Decoder`-only architecture (Radford et al., 2018; 2019), as it is focused on next token generation only.

[2] Based on Google Scholar, accessed June 2026

As our goal of next-event prediction requires us to generate new steps, our model can be a `Decoder`-only network as well. We will therefore focus on presenting the architecture structure of that submodule.

The first step is to convert the input sequence tokens to vectors of size d_{model}. With a context window size d_{ctx}, which is the maximum number of tokens processed at the same time by the model, this conversion translates our sequence of tokens into a two-dimensional tensor of size $d_{\text{model}} \times d_{\text{ctx}}$. This transformation is performed by a trainable linear layer, essentially the index of the tokens in the vocabulary to the lower dimension, *embedding* it. Both d_{model} and d_{ctx} are *hyperparameters* of the architecture, as are further variables written as d_{param}, which must be chosen before training.

Next follows a *positional encoding*, where fixed geometrically decreasing frequencies are added to the input embeddings to encode the position of the token within the sequence. As no further weight is given to the explicit original sequence order in the following steps, this encoding provides the only way to distinguish two tokens' distance in the upcoming layers.

The now properly embedded sequence is now passed through d_{layer} repetitions of the transformer blocks. They each again consist of three sublayers, each connected by a layer normalization. Assuming input x to a sublayer and $\text{Sublayer}(x)$ the function performed by the sublayer, the output is $\text{LayerNorm}(x + \text{SubLayer}(x))$, to keep the output stable without any unexpected outliers complicating the gradient descent during training.

Two of the sublayers are *Multi-Head Attention Layers*. Here, the current embedding is first multiplied by $d_h \cdot 3$ linearly learnable parameter matrices $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ with $1 \leq i \leq d_h$. The results are triples of the form (Q_i, K_i, V_i). In more optimal implementations, this computation does not need to perform $3 \cdot d_h$ matrix multiplications, but instead combined into one larger matrix multiplication. The triples are fed into the *scaled dot-product attention mechanism*, defined as

$$\text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{\text{mask}(Q_i K_i^T)}{\sqrt{d_k}}\right)V_i$$

This computation can be intuitively understood as follows.

1. Combining Q_i and K_i: Q_i represents a certain learned query, essentially encoding which part of the embedding is interested in getting certain information of other tokens. K_i highlights positions in the embedding, that match this information of the query Q_i. By multiplying them, one combines the interested embedding with the tokens that contain information.
2. Lowering the magnitude of the values in the combined matrix by dividing by $\sqrt{d_k}$, such that the softmax function has smoother gradients.

3. Masking out all fields that try to let tokens refer to tokens after them, by setting the matrix upper right diagonal to $-\infty$.
4. $\mathrm{softmax}$ itself performs a rowwise normalization, such that all rows represent a random distribution, i. e. summing up to 1 while keeping the relative magnitude information. Values of $-\infty$ result in 0.
5. Multiplying by V_i: V_i contains the embedding of the information that is present, if the query and key match. Thus, the multiplication linearly scales that information within the embedding.

The resulting matrices are of the shape $d_{\text{ctx}} \times d_v$. They are now concatenated into one $h \cdot d_{\text{ctx}} \times d_v$ matrix and multiplied by a last parameter matrix $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

The third sublayer of the transformer block is a simple 2-layer fully connected feed-forward network with a ReLU activation function. The input and output layers have dimension d_{model} and the inner layer $d_{\text{dim\_ff}}$.

After the transformer blocks, we need to extract the next token. Similarly to the initial embedding, we now need to map the embedded d_{ctx} d_{model}-dimensional vectors back to our tokens. We again apply a learnable linear layer to the embedding, resulting in vectors the same size as the vocab with all tokens. After normalization, the vector at position i contains a probability distribution over the token at position $i + 1$.

We acknowledge that there are further optimizations to this architecture since the original release, mostly on performance and resource usage (Kwon et al., 2023), and that there are multiple adaptions to other domains such as image processing (Han et al., 2021). Due to our limited dataset and resulting small parameter set, model performance and resource consumption should operate on a scale that is not a concern for us in the first place.

2.3.2 Model Training

Training not only consists of fitting our parameters to the training token sequences. Before training, we need to perform a hyperparameter selection.

Besides the model parameters from above, we need to also select parameters for the deep learning optimizer, in this case the AdamW optimizer (Loshchilov & Hutter, 2019), which builds upon the Adam optimizer (Kingma & Ba, 2017), improving its generalization performance by decoupling the weight decay. Parameterwise it takes a learning rate d_{lr}, the weight decay, two running average parameters β_1, β_2 and a numerical stability parameter ε.

Using cross validation on k-fold splits of the training data, we search through a subset of hyperparameters. These hyperparameters are based on the model defaults and changed according to the training data size we provide.

2.3.3 Technical requirements for Process Prediction

Some approaches to apply the transformer architecture rely on natural language to express the process steps (Rebmann et al., 2025). However, this approach requires the model not only learning things about the process, but also understanding the language.

Our approach to use the transformer architecture therefore relies on training a *smaller*[3] model following the original transformer architecture from scratch.

The transformer architecture matches the requirements of our process prediction problem on multiple levels. From a general perspective, it is designed to process and output sequences of elements, just as in process prediction past events are considered to predict future events. The initial embedding of events from tokens to lower-dimensional vectors allows the model to learn a more compact representation of events, instead of relying on a one-hot encoding of tokens. This allows to keep the model size small even with a large number of different events, by embedding similar events into similar vectors.

The characterizing attention mechanism allows the model to learn causal relationships between elements in the sequence. For the next event, relevant previous elements can be attended to, while irrelevant elements are ignored. This is especially important when modelling sequences that are only partially ordered, as in the case of concurrent systems.

Additionally, the transformer architecture is highly scalable, from a small model with only a few layers and attention heads to represent a small process without parameters and few causal relationships, up to models with billions of parameters to represent highly complex processes on huge datasets. Even though we are using our technique on an experimental scale, the scalability allows the model architecture to be applied to a wider range of process prediction problems.

Finally, the output of the model includes the *probabilities* of the different next tokens. One can either just choose the one with the highest probability, or sample off of the then provided distribution. The probabilities can also be used to express a confidence measure of the prediction, or to provide multiple possible next steps in case of non-deterministic processes.

[3] Compared to a typical natural language model by the number of learnable parameters

For our Fischertechnik APS, we are need to provide our encoding of the Heraklit steps into *tokens*. We choose to make **every possible step its own token**, as our case study does not require taking in parameters for steps.

In case parameters are necessary, we considered two approaches. Parameters can be encoded into a sequence of tokens, creating special meta-tokens to describe the start and end of parameter sequences. The sequence of tokens to encode our starting example of start of the production of a product `A` with quantity `3` could be encoded as the input sequence [`Produce`, `ParamStart`, `A`, `3`, `ParamEnd`]. The model would then need to learn the semantics of the parameter sequence, which could be difficult for larger parameter sets, also ensuring a complete parameterization and correct order of parameters depending on the step. Formally, a step `S` with parameters $p_1, ..., p_n$ is then represented as a sequence of tokens [`S`, `ParamStart`, p_1, ..., p_n, `ParamEnd`], where the order of parameters is fixed based on the step.

Alternatively, parameters can be represented by creating multi-dimensional input- and output tokens that are parsed as parameters. Latter approach could be implemented by multiple parallel inputs, where each parameter type sits at a fixed input position, which is zero-padded if not used for a specific action. Here, the upper example could be represented by a tuple of the form (`Produce`, `A`, `3`), where the first element is the action and the following elements are the parameters.

This approach only works a small number of different parameters over different steps, as we need to be able to represent all possible parameters of all possible steps in the same vector space. For example, if `Produce` has two distinct parameter types, one for the item and one for the item parameter, and `Combine` has two distinct parameter types, our complete input to our model would always need to be a tuple of the form (`Step`, `ProduceItem`, `ProduceQuantity`, `CombineParam1`, `CombineParam2`) independent of the specific step, where the last two elements are zero-padded for all steps that are not `Combine`.

Formally, given the set of all steps $S = \{s_1, s_2, ..., s_m\}$ and parameters $p_{i,1}, ..., p_{i,k_i}$ for each step $s_i \in S$, the final input vector would be of the form $(s_i, p_{1,1}, ..., p_{1,k_1}, ..., p_{m,1}, ..., p_{m,k_m})$. For an individual step s_i, the input vector would be $(s_i, 0, ..., 0, p_{i,1}, ..., p_{i,k_i}, 0, ..., 0)$. By sharing parameter positions across multiple steps, the dimensionality of the input vector can be reduced, however without enough parameter sharing, the sparsity of the input vector increases with the number of different steps and parameters. This approach is therefore not feasible for a large number of different steps with different parameter types, as the input vector would grow in size and sparsity.

Further details on Fischertechnik APS specifics and the training of our model are discussed in Chapter 4.

3

Heraklit Modeling

This section marks the beginning of our case study, showcasing the approach of using Heraklit and the Transformer architecture to perform next-step process prediction.

We want to evaluate our process prediction technique on the Fischertechnik **Agile Production Simulator** (Fischertechnik, n.d.). As required by our definition of what a correct prediction is (*Def. 12*), we will first need to translate events of the APS into Heraklit (Fettke & Reisig, 2021) steps. These steps should crucially allow the modeling of concurrency within the system by only modeling the causal relationships of events.

While some domain knowledge is necessary for this step, still only step modules need to be created, not full system process models. As these steps are predicted by our technique, having a clear understanding of what each step represents is useful during further evaluation.

We can split and group the steps by the relevant factory modules:

- Automated Guided Vehicle (**AGV**): It autonomously transports workpieces between factory modules.
- High-Bay Warehouse (**HBW**): It serves as a storage system to the factory, designed with an automatic retrieval.
- Delivery and Pickup Station (**DPS**): It serves as the point of input and output of the factory. New workpieces are *inserted* here, processed workpieces are *shipped off*. The included NFC reader starts the tracking of workpieces across the factory.
- Drilling Station (**DRILL**): It performs a simulated drilling operation on the workpieces, picking them up from and dropping them onto the AGV.
- Milling Station (**MILL**): It performs a simulated milling operation on the workpieces, picking them up from and dropping them onto the AGV.
- Quality Control with AI (**AIQS**): Checks the quality of a workpiece using a camera and color sensor. If a failure occurred - represented by an erroneous print on the workpiece - the workpiece is discarded.

- Central Control Unit (**CCU**): It is the central controller of the factory. While it has no physical representation in the factory, it is the centralized decision maker of the factory, synchronizing the different distributed modules. Thus, for our modeling purposes, it will be the glue that connects the modules together.

3.1 AGV

We start by modeling the AGV. It is the main source of interaction in the APS, however it can only perform one action by itself, which is moving from one processing module to the next. As we want to have one token per step, we need to create multiple **move AGV** steps.

For each module `M1` $\in \{\text{DPS}, \text{HBW}, \text{DRILL}, \text{MILL}, \text{AIQS}\} := M$ we need to have a move step to each module `M2` $\in M \setminus \{$`M1`$\}$. Thus, in the following step module template, we get all possible **move AGV** steps by instantiating `M1` and `M2` with all possible combinations.

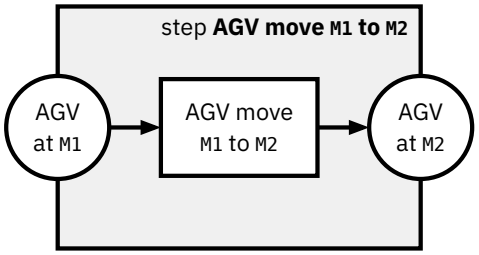


Figure 3.1: *AGV step template*

A keen reader might realize that this could be modeled using a parameterized module, an advanced HERAKLIT concept not introduced in Chapter 2. As no further parameterization was necessary in the remaining modeling, the introduction, definitions and complexity can be reduced by showcasing the template steps instead. For the sake of completeness, the step module as a parameterized module is shown below.

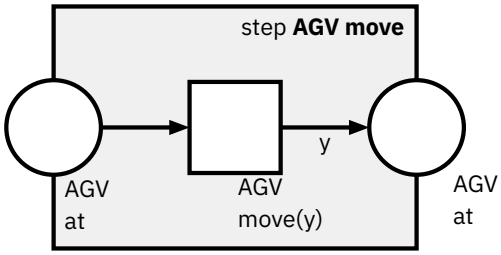


Figure 3.2: *Parameterized AGV step*

This type of modeling requires defining variables and domains as

```
variable
y: process-module
```

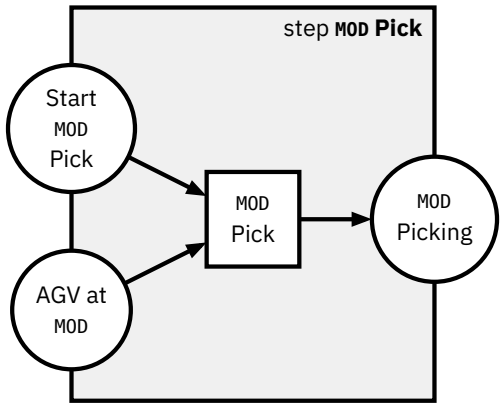
```
domain
process-module: {DPS, HBW, DRILL, MILL, AIQS}
```

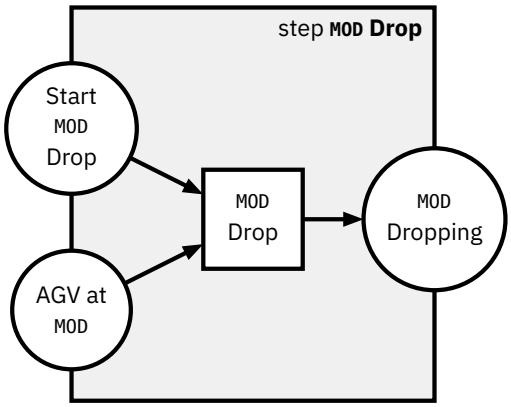
Important to notice is that depending on which **AGV move** step is taken, only certain step modules depending on the AGV position can be composed afterwards. This however also ensures that modules can depend on the AGV being at their module, locking them at that position by temporarily consuming the token at the `AGV at M1` place, with `M1`$\in M$.

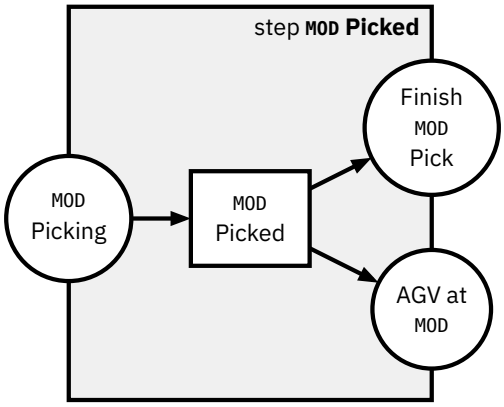
3.2 Picking and Dropping

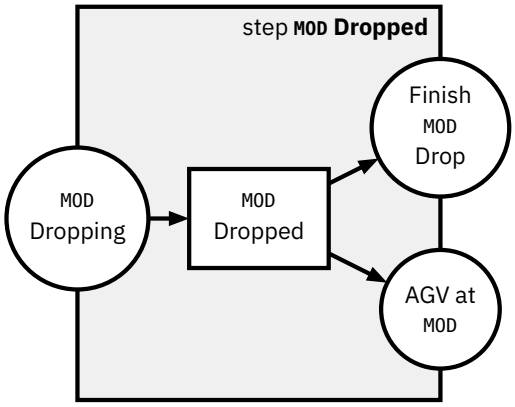
All other modules need to physically interact with the remaining factory by picking up workpieces from the AGV or dropping workpieces onto the AGV. This workflow is generally the same over all modules, thus to reduce wasted space, we again fall back to modeling these steps using the following template, with

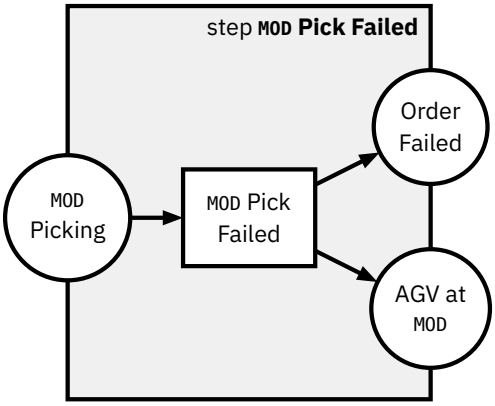
`MOD` $\in \{\text{DPS}, \text{HBW}, \text{DRILL}, \text{MILL}, \text{AIQS}\}$











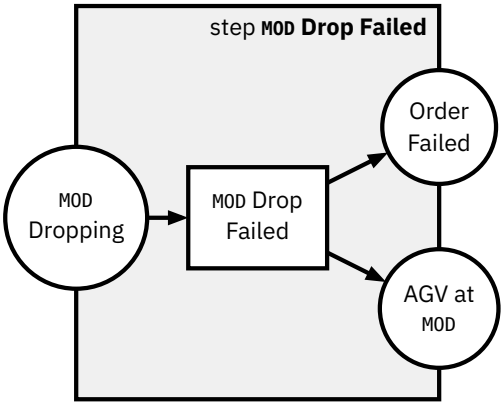


Figure 3.3: *Pick steps template*

Figure 3.4: *Drop steps template*

Two points should be highlighted:

1. We are modeling failure modes. In case the action fails, the respective `Pick Failed` or `Drop Failed` can be composed instead of the `Picked` or `Dropped` successful counterpart.
2. The AGV is not able to move while the picking or dropping action is performed, as it consumes the token at `AGV at MOD` token.

Next we will look at the individual process module actions.

3.3 HBW

The High-Bay Warehouse actions only consist of picking up workpieces from the AGV and putting them into storage, and of dropping workpieces from storage onto the AGV.

These might seem like actions that require a lot of control, first due to needing to select what workpiece should be extracted from storage, and second due to storage management itself.

However, the Fischertechnik simplified both control challenges by

1. only looking at the current running order that the pick or drop action is associated with, figuring out what color it refers to and then
2. performing a *First-In First-Out* (FIFO) storage policy for all colors, keeping the mapping of colors to ordered storage slots persistent.

Thus, when executing a pick or drop, it can do so without any further information. We decide to not try to model the internal (unexposed) storage logic here, as no events are emitted through the MQTT logs and therefore no step can be mapped to it. The generic `MOD Pick` and `MOD Drop` actions defined in Figure 3.3 and Figure 3.4 can be reused here.

A successful HBW pick run can be seen in Figure 3.5, a failed HBW pick in Figure 3.6.

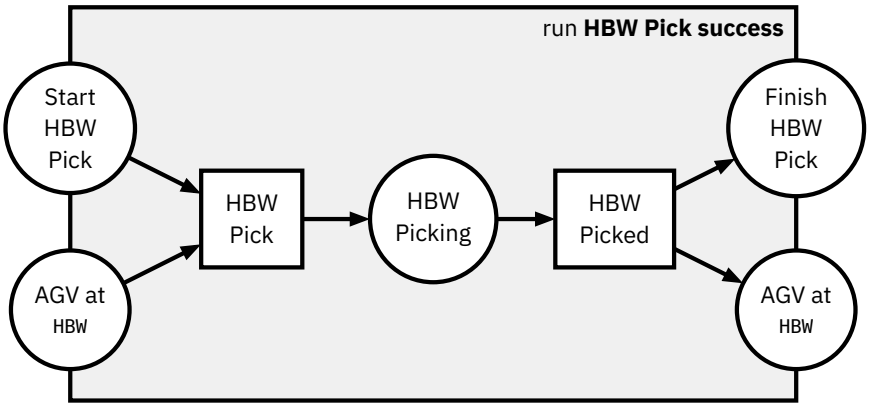


Figure 3.5: *HBW Pick run (Store into HBW)*

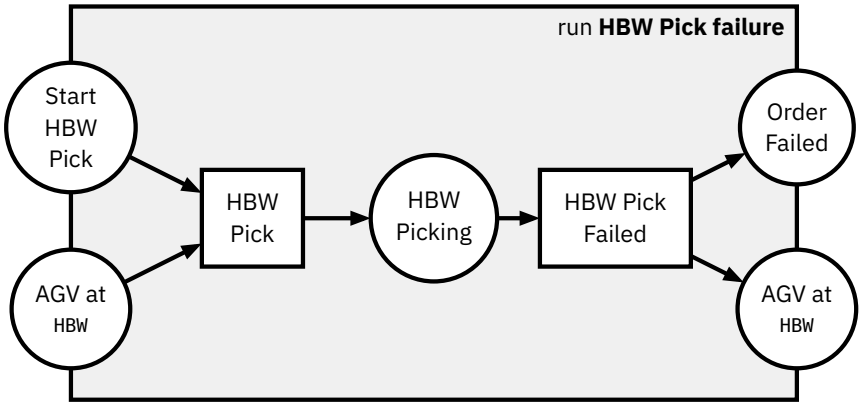


Figure 3.6: *HBW Pick run failure (Store into HBW)*

They are clearly defined by composing the singular pick steps:

$$\text{HBW Pick success} = \text{HBW Pick} \bullet \text{HBW Picked}$$
$$\text{HBW Pick failure} = \text{HBW Pick} \bullet \text{HBW Pick Failed}$$

These runs are exemplary for all module `Pick` and `Drop` runs, as they always follow the same structure.

3.4 DPS

The DPS is responsible for the insertion of pieces into the factory and shipping them out of the factory. The piece insertion into the factory is represented by a `DPS Drop`, dropping the workpiece onto the AGV from the loading bay. The shipping out of the factory is represented by `DPS Pick`, picking the workpiece up from the AGV and dropping it off at the loading bay.

Both actions are reading and writing to the NFC tag on the workpiece, if present, providing a history of the piece across the factory. Similar to the physical actuator control done by the Siemens SPS, we choose not to model the NFC tag explicitly, as the prediction will be on a higher level of abstraction. Thus, we can again re-use the template for Picking and Dropping given in the templates in Figure 3.3 and Figure 3.4.

Notably, the `DPS Drop` step is the only step that can introduce a new workpiece into the factory. New workpieces are generally first stored into the HBW and then extracted again with an order, even if an order for that piece is already present.

The failure case of a `DPS Drop` could require different processing than other dropping steps, as this is the only action where a workpiece might not have an order associated to it yet. However, as we are only processing single workpieces at a time in our dataset, the order failure case is an appropriate failure model, as either way the processing of that workpiece ends there.

We do not need to introduce any additional new steps, as again the abstraction given through the MQTT traces does not provide further details.

3.5 DRILL

Like all other physical modules, the drill process module has to interact with the AGV to process workpieces. Therefore the `Pick` and `Drop` template is defined for it as well.

Additionally, this is the first module to perform a certain module-specific action, observable via the MQTT logs. This unique action is simulated drilling of a hole into the workpiece, which can again succeed or fail. The matching HERAKLIT step definitions can be seen in Figure 3.7. Notice that

1. We require a workpiece to be picked up from the AGV to be able to start the drill action by consuming the token at the place `Finish DRILL Pick`.
2. We do not require the AGV to be at the drill module during the drilling steps, opening up the future possibility of having multiple modules process workpieces simultaneously within the same model, as the AGV can drive to a different station during processing at the drill.

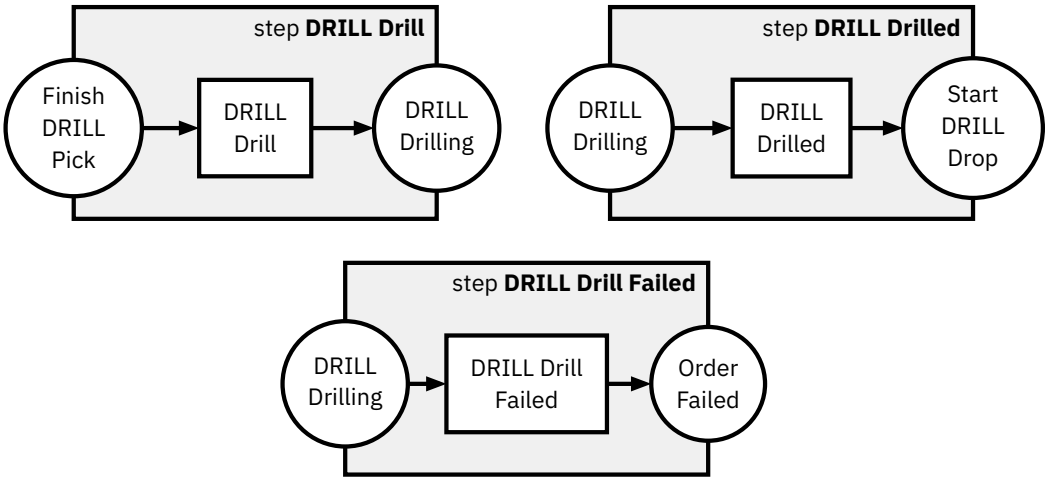


Figure 3.7: *DRILL Drill steps*

Both of these properties can be re-discovered in the following module-specific step-definitions.

While there is a parameter for the drilling duration defined for each workpiece color, we decide to not model it for two reasons. First, the parameter is constant and thus just belongs directly to the drilling operation of that color. Second, even assuming it was not constant, there is no causal relationship to be learned as to why the duration should be different. The Fischertechnik APS provides no simu-

lated tool wear or different operation durations for simulated broken or working workpieces.

We again provide a successful run in Figure 3.8 and a failed run in Figure 3.9, composed of:

$$\text{DRILL Drill success} = \text{DRILL Drill} \bullet \text{DRILL Drilled}$$
$$\text{DRILL Drill failure} = \text{DRILL Drill} \bullet \text{DRILL Drill Failed}$$

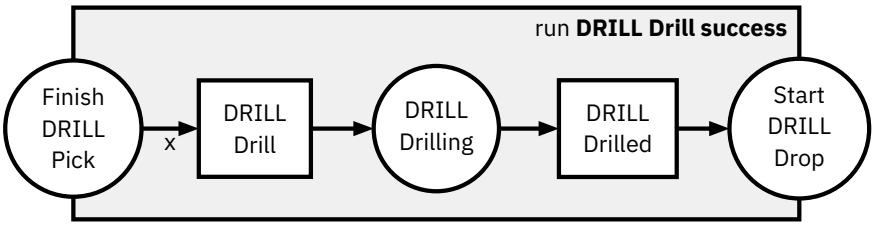


Figure 3.8: *DRILL Perform Drilling run*

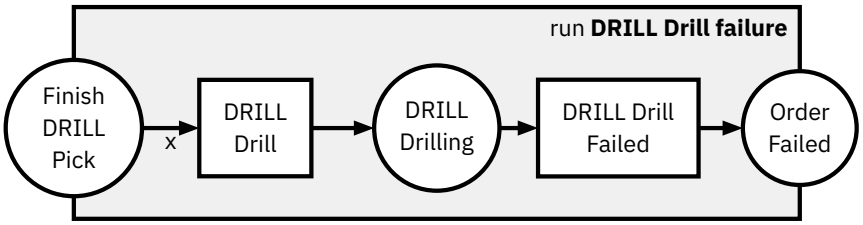


Figure 3.9: *DRILL Drilling Failure run*

3.6 MILL

Similar to the drill, the mill process module performs a simulated action, but it is an operation to simulate milling a groove into the workpiece instead of the drilling operation. It again also contains the template `Pick` and `Drop` steps.

The steps are defined identically modulo renaming in Figure 3.10. Thus, the same properties hold and the runs can be composed in a similar fashion as in Figure 3.8 and Figure 3.9.

3.7 AIQS

The last physical module is the AIQS, which provides a quality control checkpoint for all orders. To simulate failure within the APS, workpieces inserted into the factory at the DPS are designated `fail` or `pass` pieces, determined by a print on top of the piece. Using a camera and a color sensor, it can detect these faults and then discard faulty pieces into a trash chute, while passing good pieces on.

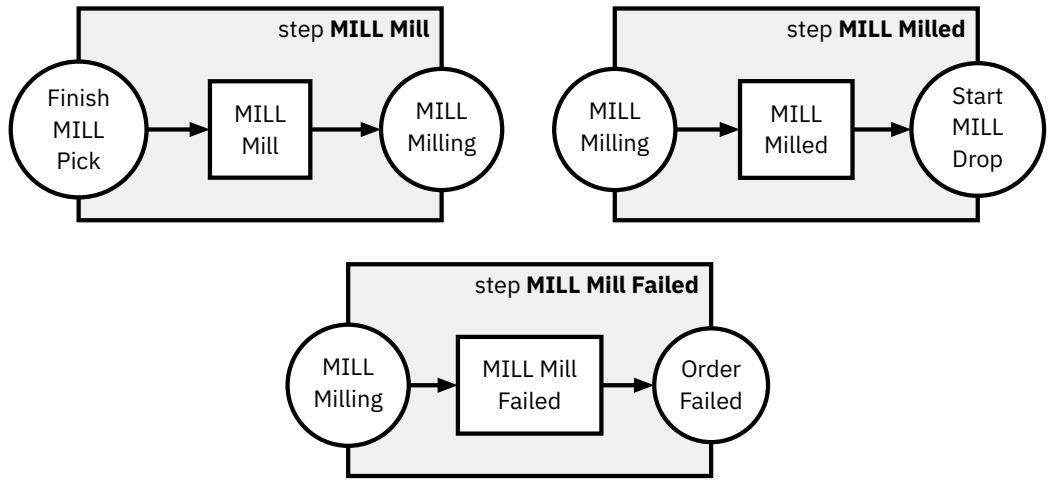


Figure 3.10: *MILL Mill steps*

Besides the `Pick` and `Drop` steps, the AIQS needs to perform this quality check action. The matching steps are defined in Figure 3.11.

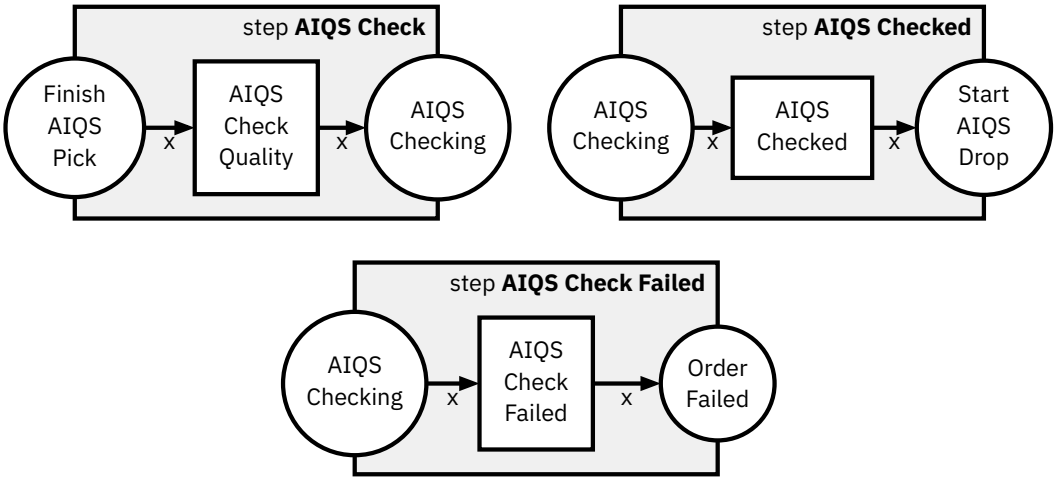


Figure 3.11: *AIQS Check Quality steps*

In the Fischertechnik APS, only the AIQS is concerned with the failure of a piece, all other pieces are processed based on just the color. This makes the behavior of the next step after a started quality check hard to predict: The piece must either fail or pass, but the previous process execution provides no indication of whether the workpiece “processed” will result in a failure or not. This is a shortcoming in the simulation of the APS, as especially tool wear and processing durations could be exploited to simulate a failing processing module on a piece, such that a prediction has some grounds to base its decision on.

We will later see that this results in missed accuracy of our prediction model.

A composed success run of the AIQS can be seen in Figure 3.12, the failure in Figure 3.13.

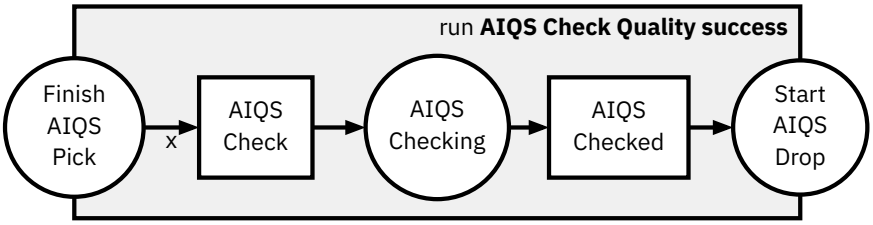


Figure 3.12: *AIQS Check Quality run*

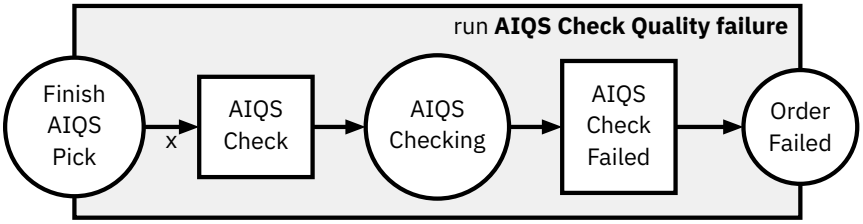


Figure 3.13: *AIQS Check Quality failed run*

3.8 CCU

The CCU is the heart of the factory. It controls the interactions between the modules, taking the decisions on where the AGV should go and interacting with the factory order system via the UI.

Our goal is to model the steps extractable from the Fischertechnik MQTT Logs. These control steps, deciding what module action happens after the next, are **implicit** or **invisible** control. There is no control action token to be found in the MQTT logs that clearly defines *after action X perform action Y*, the control can just be inferred from the interactions between the modules. This means that the CCU steps will and cannot be present in the prediction tokens, as they do not exist within the logs. However, we still need to model some control system, as the steps of different modules are not composable at the moment.

For a potential synthetic generation of runs modeling the different variations of runs, e.g. depending on the color of the workpiece, one could argue that these control steps must be meticulously designed, including the order of stations per workpiece type. This would imply pre-modeling a specific order of module actions into the steps via the connecting places. An example can be seen in Figure 3.14. Here we provide the fixed connection of a `MILL` step to the `DRILL` step.

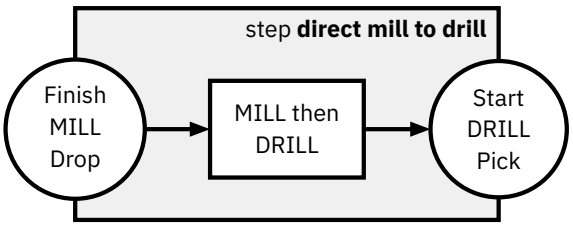


Figure 3.14: *Direct Module Interaction Modeling Approach*

This approach has multiple downsides. We trade the increased detail for **decreased flexibility**, as changes in the configuration for certain workpieces would require a new HERAKLIT step model. More importantly though, the tools to validate model outputs would need to be capable of handling an **exponential number of steps** to support all different process configurations, with increases in modules, and again exponential growth with length of runs. This is due to these control steps not being present within the logs, so all matching control steps must be tried to be appended at any point of the validation, creating a huge search tree for validation.

We therefore decide not to model all the configurations explicitly. Instead, we only restrict our model to allow one module action to take place at a time. While this might seem counter-intuitive when looking at the distributed factory setting, here we are only looking at the factory execution from the perspective of a singular workpiece. Since all processing parts of a singular workpiece are always only present in one processing module's action, these actions do not need to be able to run concurrently.

Technically, this restriction is applied by creating a global shared place called `Next Module Ready`. Whenever a module wants to start, it needs to consume this place, whenever it is finished, it will fill the place again. Following the example from before, this creates the new steps in Figure 3.15.

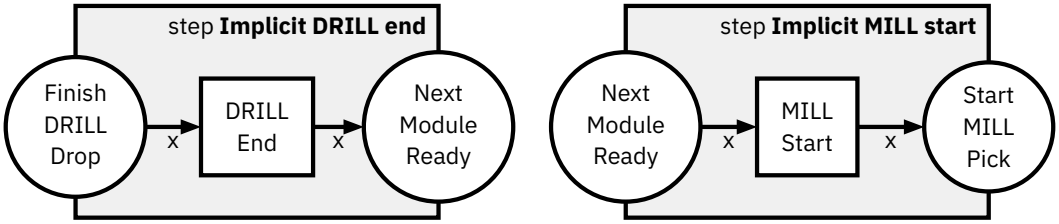


Figure 3.15: *Implicit Control of Mill to Drill steps*

This new design does not directly solve the issue of these control steps missing in the logs, but provides a simple solution. By composing DRILL Dropped • Implicit DRILL end and Implicit MILL start • MILL Pick we create a run module that essentially just changes the interfaces of the module steps to have the `Next Module Ready` place instead of their respective `Start` and `Finish` places. These newly composed models can be then again composed directly via the `Next Module Ready`.

As we know that before and after all module actions their steps will need their corresponding `Implicit` step, we can pre-compose the control and module steps when talking about the actual logs.

For example, if the logs contain the steps

`DRILL Dropped` → `MILL Pick`

we can instead interpret this as

`DRILL Dropped` → `Implicit DRILL end` → `Implicit MILL start` → `MILL Pick`

as the implicit steps can be directly inferred.

At this point, one might wonder why we have not directly put the `Next Module Ready` step into the module steps. The reason for this decision is that we can keep the level of detail on the module basis, while essentially just having an interface wrapper to simplify implementation details later.

All the implicit control steps, that are implicitly composed with the respective `Start` and `Finish` steps of the processing modules can be found in Figure 3.16. Notably, the DRILL, MILL and AIQS only connect a start module on the `Pick` action and a stop module on the `Drop` action. The DPS and HBW however can independently Pick and Drop without any ongoing action in between, so both `Pick` and `Drop` actions have their own implicit start and stop control.

For the further processing we then also redefine the following step modules as runs composed with their implicit control step:

DRILL Pick := DRILL Start • DRILL Pick

DRILL Drop := DRILL Drop • DRILL End

MILL Pick := MILL Start • MILL Pick

MILL Drop := MILL Drop • MILL End

AIQS Pick := AIQS Start • AIQS Pick

AIQS Drop := AIQS Drop • AIQS End

DPS Pick := DPS Pick Start • DPS Pick

DPS Picked := DPS Picked • DPS Pick End

DPS Drop := DPS Drop Start • DPS Drop

DPS Dropped := DPS Dropped • DPS Drop End

HBW Pick := HBW Pick Start • HBW Pick

HBW Picked := HBW Picked • HBW Pick End

HBW Drop := HBW Drop Start • HBW Drop

HBW Dropped := HBW Dropped • HBW Drop End

Since runs can be composed the same way individual step modules can, we do not require further differentiation and can **assume from now on, that the starting and stopping "steps" can be composed via the `Next Module Ready` place**.

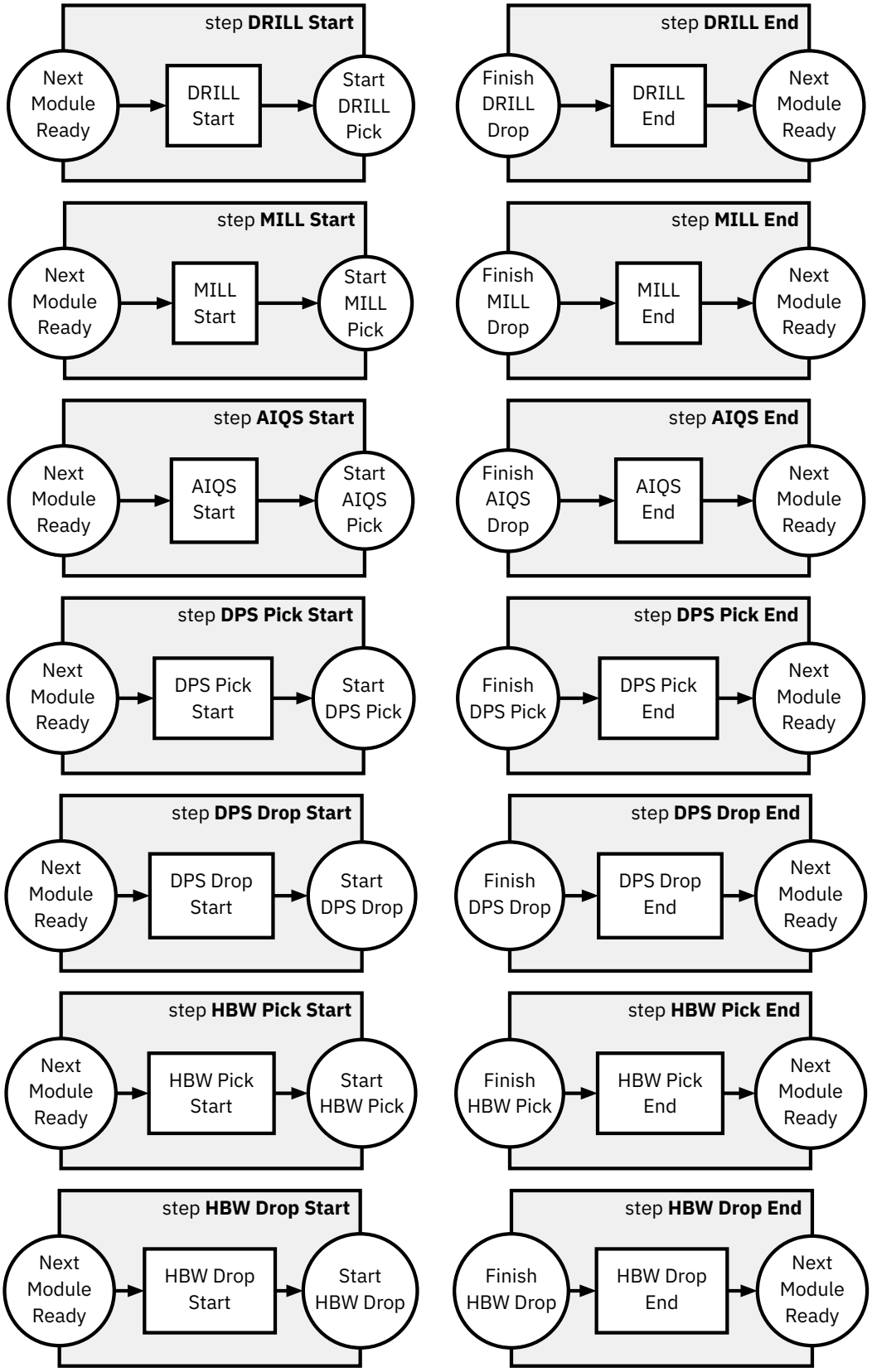


Figure 3.16: *Implicit Control steps*

4

Implementation

This chapter provides the technical details of the implementation of our process prediction technique with encompassing information on the data collection, preprocessing and on the architecture and training of the final model.

While the chapter will describe what we do and how we do it, it explicitly does not explain how to technically execute the code infrastructure. This information is available through a collection of *Markdown files* that provide a walkthrough of the project code, including the commands to execute in the command line.

For our implementation we rely on several tools:

- `python`[4]: The programming language used for the code infrastructure and scripts.
- `uv`[5]: A Python project and package manager handling the other listed dependencies and tools.
- `pytorch`[6]: An optimized python library for deep learning. It supports training on CPUs and GPUs, including integrated GPUs such as ones used in Apple Silicon processors. We use it to build our models by using integrated base models or layers, and to perform the training.
- `numpy`[7]: An optimized python library for array programming, such as tensors. This is also implicitly used by `pytorch`, and we use it to interact with data to and from `pytorch`.
- `graphviz`[8]: A library for a graph drawing software for python. We use it to visualize Heraklit runs.

The implementation was tested on both *macOS 15.7* and *Ubuntu Linux 24.04*. The hardware used for training and evaluation consists of an *Apple MacBook Pro* with an *M1 Pro* chip and *32GB* of memory.

[4] https://www.python.org *last accessed: 11.06.2026*
[5] https://docs.astral.sh/uv/ *last accessed: 11.06.2026*
[6] https://docs.pytorch.org/docs/2.12/index.html *last accessed: 11.06.2026*
[7] https://numpy.org *last accessed: 11.06.2026*
[8] https://graphviz.readthedocs.io/en/stable/ *last accessed: 11.06.2026*

4.1 Data Collection and Preprocessing

In a first step, the data needs to be *extracted* from the Fischertechnik APS. As described in Section 2.2.2, MQTT consists of *publishers* and *subscribers*, that communicate via a broker. We add an additional client to the broker by connecting a computer to the APS network, which runs a script that subscribes to all topics, using a wildcard subscription ("*"). It will therefore be sent a copy of every message in the APS.

Whenever it then receives a message, the message is decoded, combined with the MQTT topic and the timestamp of the receiving computer, then dumped to a file in *JSON* format. This format is chosen as the messages sent through the broker are all in a *JSON* format already.

We **captured** the data of 10 runs. Each run consists of a single workpiece being processed throughout the APS, from the insertion to the factory at the DPS up to either the successful delivery at the DPS or a failed quality control.

Before any further processing, all JSON log files are read, checked for valid JSON syntax and then **combined** into one dataset of around 60MB . Here we also extract the color of the processed piece and pass it into the metadata.

Next, we perform some **preliminary filtering**, essentially removing the messages regarding two topics: The first one contains the raw image data of the camera mounted on the APS, wasting space in the logs. The second one removes a single action that makes sure a status LED is blinking. This removes 5298 and 1996 messages respectively, reducing the dataset size to only around 14MB.

The camera data could be interesting for future work, so we decided to still collect it, even though we will not make any use of them here.

The next step consists of the proper **preprocessing** of the trace data. Here, we analyze the messages themselves to **extract the tokens**, which represent the Heraklit steps modeled in Chapter 3. Note that these extracted steps are assumed to already be implicitly composed with the control steps of Figure 3.16.

We can distinguish 3 different message types relevant to our modeling based on the MQTT topic:

- `module/v1/ff/<SERIAL>/order`: This topic contains messages from the CCU to the respective module with the serial number `<SERIAL>`. These **order messages** contain the instructions to a module to start a certain action.
- `module/v1/ff/<SERIAL>/state`: This topic contains messages from the module with the serial number `<SERIAL>` to the CCU. These **state messages** are regularly transmitted and contain the state of an action in the module.
- `fts/v1/ff/<SERIAL>`: This topic contains all messages related to the **AGV** with serial number `<SERIAL>`. It is again split into `/order`and `/state` messages, however, for our modeling only the AGV *orders* are relevant.

We start by creating a mapping of serial numbers to module types, to handle the different module actions. We can then usually extract the start of a new action and thus the `Start X` step from our model through the module **order messages**. For the result of an action, we need to listen to the **state messages** until we need to find one that describes the status of the module as finished or failed to extract the next correct step.

Due to the QoS levels of MQTT and the retained messages of the APS MQTT broker, we need to apply some heuristics to filter out duplicate or old messages. This could have been partially avoided by also writing the MQTT `dup` flag of messages into the logs. However we would still need to figure out whether we received a message before or not regardless.

Notably, a different modeling of our steps would change the MQTT processing in a significant way. This could be supporting duplicate messages in the modeling itself, changing the level of detail of each step or changing the step definitions themselves. If duplicate messages are supported in the steps, there is no need for duplicate filtering. With increased detail within a step or new step definitions, more metadata can be extracted from the MQTT messages themselves, such as the workpiece ID, intermediate processing states or the machine configuration. With more details comes the need to be able to predict this detail of the next steps as well.

Possibly, increased levels of detail would also require a different encoding of the tokens. Different encodings are discussed in Section 4.3.

The then **extracted tokens** are written to a new processed JSON file. We additionally add some metadata to the tokens for analysis purposes, such as the time the message was sent from a module, the time it was received, the module serial number and message IDs.

The 10 extracted tokenized traces consist of an average of 23.1 steps, with a minimum of 19 and a maximum of 31 steps. It consists of 4 traces of white workpieces, and 3 traces of red and blue workpieces each. For each color, it contains

successful and failed runs. Two of the traces are duplicates of other traces, even though their MQTT logs are different. This is due to their extracted relevant steps are in the same order, even though other messages might be in different orders or different in their metadata. While duplicates would usually be removed, we treat our traces as separate due to differences in their raw MQTT data.

Lastly, we perform a random split of our tokenized traces into 7 training and 3 validation traces. Concerns regarding dependent traces within training and validation datasets as presented by Pfeiffer et al. (2025) are not relevant, as all our executions in the APS are independent of one another. This would change if we run multiple APSs in parallel or have multiple workpieces processed at the same time and extract the traces per workpiece.

The two tokenized trace sets are then written to two files. We ensure that from now on the model training process does not interact with the validation data.

4.2 Heraklit Prefix Checker

In *Def. 12* we define what constitutes a correct prediction based on potential next steps (*Def. 11*). This definition, while easily understandable, has the caveat of not being written in a computationally simple way.

Following the definition directly, given a *reference* Heraklit run, and a second *checker* Heraklit run, to check whether the *checker* run is a prefix of the *reference* run, we would need to model all possible suffixes and then check whether they are equal.

However, by the composition calculus (*Def. 4*) we know that the two sequences of compositions are equal and represent the same run, if composition produces the same graph. Following from that, a *checker* sequence of compositions is a prefix of a *reference* sequence of compositions, if the composition graph of the *checker* is a **graph prefix** of the *reference* composition graph.

We define A as a **graph prefix** of B , if the *initial nodes* (here: nodes without incoming edges) of A are a subset of the *initial nodes* of B and if A from these *initial nodes* is a subgraph of B.

We limit our initial nodes to only contain unique labels, as otherwise all possible initial node mappings must be explored, which is computationally expensive. This theoretical limit does not provide any limitations in practice, as proper runs on a single workpiece never contain multiple initial steps of the same type.

We built a tool and python library around this concept, which processes two sequences of predefined steps and checks, whether one is the prefix of the other.

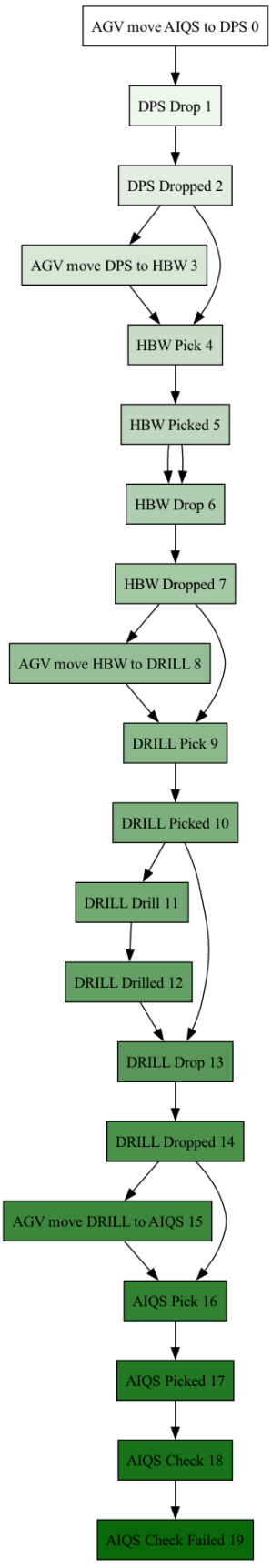


Figure 4.1: *Example Composition Graph*

To ensure the correctness of the tool, an extensive test suite is created and passes. This tool's code is additionally published on GitHub[9] to allow using it for further Heraklit-based process prediction projects.

The tool also contains a neat feature to display the composed run graphs of the two compared runs, leaving out the Petri net places. This visualization can help to understand the composition and why predictions might be deemed incorrect. A sample visualization of a Fischertechnik APS can be seen in Figure 4.1. It shows a run with a failed quality control. Note how steps that depend on multiple places have input edges coming from the steps that produced the corresponding place.

Thus, given a reference run r, a prefix p of it and a prediction n made by our model, we check the correctness of this prediction by using this tool, asking whether $p \bullet n$ is a prefix of r.

4.3 Model Architecture

For next-event predictions, we use the Transformer architecture as presented by Vaswani et al. (2017) and explained in Section 2.3.

As tokens, we use the steps defined in Chapter 3, plus some additional tokens:

- `<PAD>`, `<BOS>`, `<EOS>`: These meta-tokens are required for the training of the transformer. `<PAD>` is used to allow training on sequences shorter than our context window, padding the remaining window out. `<BOS>` and `<EOS>` mark the *beginning* and the *end* of the sequence of tokens, letting the model stop its prediction when the process is over.
- `<COLOR RED>`, `<COLOR WHITE>`, `<COLOR BLUE>`: These tokens are inserted at the beginning of a sequence to let the model know about what kind of workpiece is currently processed. This allows learning the different process configurations for different workpiece colors without creating much overhead.

We choose not to encode the workpiece color directly into all tokens, as each step would need to be encoded for all colors, tripling the number of tokens. Most tokens would then need to learn a very similar behavior within the model for each color. Only at the points of differently defined process behavior, these tokens would need to show different model behavior. By only encoding the color at the beginning of the sequence, we leverage the attention mechanism of the transformer to learn to refer to the color token when needed. For other tokens, the behavior is simply *shared* for all colors, without the need to refer to the color token.

Additionally, by *sharing* the token between colors, cross-color learning is possible. Due to our limited dataset, this is especially helpful, as the model can infer the

[9] https://github.com/Niggelgame/heraklit-equiv-checker/ *last accessed 11.06.2026*

shared behavior from all training runs. With separate tokens for each color, the model would not necessarily learn the shared behavior, but could treat each color separately, thus requiring more training data to learn the same behavior.

As discussed in detail in Section 2.3.3, instead of having a one-to-one encoding of steps to tokens, we could have used a multi-token encoding for each step. Especially with possibly increased detail in the steps, see Section 4.1, this could be a reasonable approach to avoid token count explosion, especially with larger classes of step parameters. The main complexity of multi-token encodings would be to ensure that the model not only learns to predict the next event, but also the correct sequence of tokens for a single step to later be able to decode the parameters of a step into a single configured step. To reduce complexity, we decide not to use a multi-token encoding for our steps, especially given the choice to not further parameterize the steps.

We use an embedding layer to encode the one-hot encoded tokens into a denser vector representation, see Section 2.3. While reducing the dimensionality of the input and thus the complexity of the model, it also allows the model to learn relationships between tokens, by encoding similar tokens into similar vector representations.

The hyperparameters of our model architecture are chosen using cross-validation on k-folds of the training data, as discussed in Section 2.3. We try to provide reasonable values for each hyperparameter, then iterate all combinations of these hyperparameters and choose the best model on the basis of a loss function.

The loss function represents the sum of the embedded distances between the real next token and the predicted probability distribution of tokens.

Further training details are discussed in the following section.

4.4 Training Details

The model is generally trained in a set number of *epochs*. In each epoch, the training data is provided to the model to compute the *loss*, a metric describing a *distance* of the prediction to the correct results. The lower the loss, the better.

We use a **cross-entropy loss** function that always sets the loss to 0 for a position if the next token is a padding token. The cross-entropy loss creates strong gradients for the optimizer by relying on a negative logarithm of the probability assigned, if the prediction is incorrect, which results in exponential penalty and thus loss for incorrect predictions.

After computing the loss, we perform a `pytorch` backward computation. This computes a loss differential for each learnable parameter, thus specifying how much

the loss would change in which direction if the parameter is changed in a certain direction. This differential is then provided to the AdamW optimizer (Loshchilov & Hutter, 2019), which computes the next set of parameters for our model, hopefully lowering the loss.

During training of our model, we add a small *dropout* layer into our model after the positional embedding of our tokens. *Dropout*, as the name suggests, drops parts of the tensor it computes on. These parts are always selected randomly based on a probability defined as d_{drop}. This layer tries to regularize our model to not overfit certain parts of our embedding, as the model must rely on multiple different parts of the input. This behavior has been validated in previous work also related to process prediction (Evermann et al., 2017). Crucially, the dropout layer is disabled during evaluation of the final model by using the `.eval()` pytorch feature on the model.

This process is repeated for a fixed set of epochs, until a certain loss is reached or until not enough loss progress is achieved.

4.4.1 Selecting Hyperparameters

Before training the actual model, we need to select its hyperparameters.

The values for the different hyperparameters we search though are:

```
d_models = [16, 32]
layers = [1, 2, 3]
dropouts = [0.1, 0.2, 0.3]
learning_rates = [3e-3, 1e-3]
```

On each possible combinatorial set of parameters, we perform a **k-fold cross-validation**. This is a standard technique to avoid overfitting while just using training data. The data set is split into k equally sized groups of data, the so-called *folds*, then we train the model k times, always using $k - 1$ folds for training, and one for validation. This technique is especially relevant for the small dataset we have, as we cannot be sure that a single random split properly distributes the data into fair training and validation sets. We chose `k = 4`.

During this pre-training phase we train full models with the same number of *epochs* as the final model. However, if a model does not show loss improvements after a number of predefined cycles of training (here: 8), we stop the model evaluation here. Models stopped in pre-training early either suggest a highly performing model, that has found good parameters early on, or such a bad model configuration, that training it further probably does not produce much of an impact either.

During cross-validation we also measure the performance of the combination of most probable 3 output tokens, producing higher `top_k` performance if any one of these top 3 tokens are correct.

We then score the different configurations. We do not only consider the average model correctness, but also include a small factor of the `top_k` performance. While we want to optimize for the *one* most probable output in the final trained model, we want the architecture to support producing other sensible options. This way of scoring rewards fully correct models the most, as providing a correct top hit implies the `top_k` also contain a correct hit. But it also chooses a model that produces good overall second or third hit performance over one that only has the same top hit performance with bad second or third hit performance.

The highest scoring configuration is then selected as the one with which we then perform the training on the full data.

As we are training full models for each model configuration, this process is the most time-consuming, taking around 3 minutes on the previously described MacBook configuration. The final selected configuration is the following:

$$\begin{aligned} d_{\text{ctx}} &= 128 \\ d_{\text{lr}} &= 0.003 \\ d_{\text{weight\_decay}} &= 0.01 \\ d_{\text{dropout}} &= 0.2 \\ d_{\text{model}} &= 32 \\ d_{h} &= 4 \\ d_{\text{layer}} &= 3 \\ d_{\text{dim\_ff}} &= 128 \end{aligned} \tag{4.1}$$

4.4.2 Training Results

The final training based on the parameters in (4.1) takes 30 seconds at full load, using the integrated graphics chip. The cross-entropy loss of the final model is 0.8392.

A more interesting statistic is the top hit correctness rate of *only* 89.2%, the top three hit correctness rate being 98.4%. While this might seem surprisingly low for a model we just trained, we need to keep in mind the dropout layer we explicitly added to the training process.

In Chapter 5 we will use the split off validation set to perform some different evaluations on the now obtained model.

5

Evaluation

In this chapter we will first compare our prediction model with a random and an empirical baseline, showing that our model is able to learn about the underlying process. We then evaluate different generalization capabilities of our model.

5.1 Baseline Comparisons

We want to predict the next step of the APS process, which is processing a single workpiece. One can understand this as predicting the next step from the perspective of the CCU, as the CCU controls the full behavior we model, or from the perspective of the workpiece, as we mostly predict actions only concerned with work on this workpiece. One exception to the last point is given by potential movement of the AGV while the workpiece is processed. This step is valid, but not relevant for the singular workpiece perspective.

Given by the data collection process, the runs produced as the validation data set are already provided from this perspective. Thus, it remains to check the predictions.

In the following we will refer to our model as a function $(n_1, ..., n_k) = \text{predict}(P, k)$, where P is a run prefix and k is the number of best next steps options after P the model should output, ordered by their probability.

5.1.1 Simple Next Step Check

For each run $R = s_1 \bullet ... \bullet s_n$, where s_j is a step token, we take all prefixes $P_i = s_1 \bullet ... \bullet s_i$ with $1 \leq i < n$. We then let our model predict the most probable next step $n_i = \text{predict}(P_i, 1)$. If $n_i = s_{i+1}$, we set $c_i = 1$, else $c_i = 0$.

We calculate the *run accuracy* metric for each run as $A_R = \frac{1}{n-1} \sum_{i=1}^{n-1} c_i$, dividing the number of correctly predicted steps by the number of totally predicted steps.

As different runs have different lengths n, the total *accuracy* of our model on the validation set is calculated by an average over all run accuracies, as otherwise longer runs would be deemed more important than short ones. This should not be the case, as especially shorter runs contain the failure cases, resulting in an early process abort.

We therefore calculate the *accuracy* over our validation set of runs val as

$$A = \frac{1}{|\mathrm{val}|} \sum_{R \in \mathrm{val}} A_R$$

We still need to provide a baseline, so we measure the same metric on a random predictor, selecting a **random** next step $\mathrm{predict}'(R, k)$, drawing k distinct steps from the set of possible steps.

We also provide an **empirical** predictor $\mathrm{predict}''(R, k)$, that processes the initial training data set and extracts the number of times a token appeared in the dataset. We then order the tokens by descending count and predict the first k tokens, which are the k tokens that appeared in the training data set the most. If there are multiple tokens with the same count that, sampled, would lead to more than k predictions, we again randomly sample from that group.

The random baseline is able to achieve an accuracy of **1.79%**, meaning 1.79% of next steps were predicted correctly. The empirical baseline achieves an accuracy of **5.36%**.

Our model achieves an accuracy of **91.07%**, highly outperforming the random and empirical baseline. This is a clear indicator that our model was able to learn properties the structure of our process. This proves the model is learning about the sequential and causal relationships and not only mimicking the frequency of contributions.

It is notable that this accuracy is higher than the accuracy measured during training; we expected this behavior due to the disabled dropout layer during evaluation.

5.1.2 Top-K Next Step Check

This scenario tries to adapt to an issue found in the Fischertechnik APS simulation control: Some next steps are basically unpredictable based just on the previous execution, leaving multiple options. One highlighted example from before is the quality control. The APS does not change its behavior if the processed workpiece is destined to be failing the quality control. Thus, the predictor also cannot catch any special structure pointing towards a failure. The decision of the next step at this position can be described as non-determinism. This means that after the quality control has started, both the quality control success and failure steps are both highly probable; the predictor has no way of knowing which one is correct.

We want to define a correctness measure that allows for lenience in the prediction due to the structure described above. Therefore, in the following, we check whether one of the top k next step options ordered by their model output probability is correct instead of just the next step option with the highest probability.

We can simply adapt our definitions from the previous scenario to predict the k most probably next steps at each prefix $n_{i,1}, ..., n_{i,k} = \text{predict}(P_i, k)$. We then change the correctness measure to deem a prediction correct, if at least one option is a correct prediction, formally if $\bigvee_{j=1}^{k}(n_{i,j} = s_{i+1})$, we set $c_i = 1$, else $c_i = 0$. The accuracy measure calculations on the correctness remain the same.

With $k = 2$, we can observe a top-2 accuracy of **our model** of **100%**. The *random* baseline only predicts the top-2 events correctly with an accuracy of **5.36%**, the *empirical* baseline has a **10.71%** accuracy.

To put the prediction performance of our model into context, even with $k = 10$, the random baseline only achieves an accuracy of 21.4%, the empirical baseline 57.14%.

5.2 Generalization Performance

Pfeiffer et al. (2025) highlight the importance of generalization within the scope of next event prediction. Generalization here refers to the ability to make correct predictions for traces with unseen behavior, based on some implicit representation of the process structure within the model.

While our general baseline evaluation cases in Section 5.1 are measured on traces not seen during training, and we are using cross-validation and dropout to reduce overfitting and guide towards generalization, we want to ensure that our model can cope with unseen behavior not found within the original dataset.

We specifically want to focus on potential issues related to the transmission noise during the transmission of events to the predictor. This could be in the form of highly delayed messages arriving at an unexpected time or messages that are dropped and not received properly. Since the APS is a networked system, dropped or late messages are to be expected, thus robustness when dealing with such symptoms is desired. We will evaluate these two scenarios on our model by introducing synthetic noise in the described forms into the prefix runs passed to our model, comparing the resulting accuracy with the accuracy from our baseline.

An additional experimental evaluation to test our generalization performance is to extrapolate the type of runs we see in the training data. Instead of just processing a singular workpiece in one sequence, we perform a short analysis to see how our model performs on one long run processing multiple workpieces at once.

5.2.1 Additional Random Events

To simulate the arrival of highly delayed messages, we will insert random events into the prefix traces. The number of insertions is controlled by a parameter p, describing the ratio of number of newly inserted events to the size of the prefix event sequence. For example, with a trace prefix of length 10 and $p = 0.2$, we would insert 2 random events at random positions into the prefix trace. The random events are drawn from the set of all possible events with replacement.

To validate the correctness of the predicted step, we append the predicted next step to the *original prefix* and check this for correctness.

We differentiate between two sub-scenarios:

- Insert at all positions except the last. This ensures that the model does not get unfairly reduced accuracy, as remotely possible events inserted after the last event might get treated as the situation where multiple events between the two last steps were dropped
- Insert at all positions

We evaluate both sub-scenarios with p starting from 0 and increasing in steps of 0.05 up to 0.5.

For the first scenario the accuracy remains mostly stable, dropping slightly over the increasing p from ~91% down to ~86%. The accuracy even slightly increases in some cases around $p = 0.3$. This can be explained by the amount of events inserted into the trace: The new events create a new context for the model, which leads to a correct prediction for the original unmodified prefix, under which the model has failed before. This progression is plotted in Figure 5.1 on the left.

The performance drop in the second scenario, inserting random events being possible at all positions, worsens the accuracy much more significantly, up to only ~**43.4%** with $p = 0.5$. The progression over increasing p is plotted in Figure 5.1 on the right.

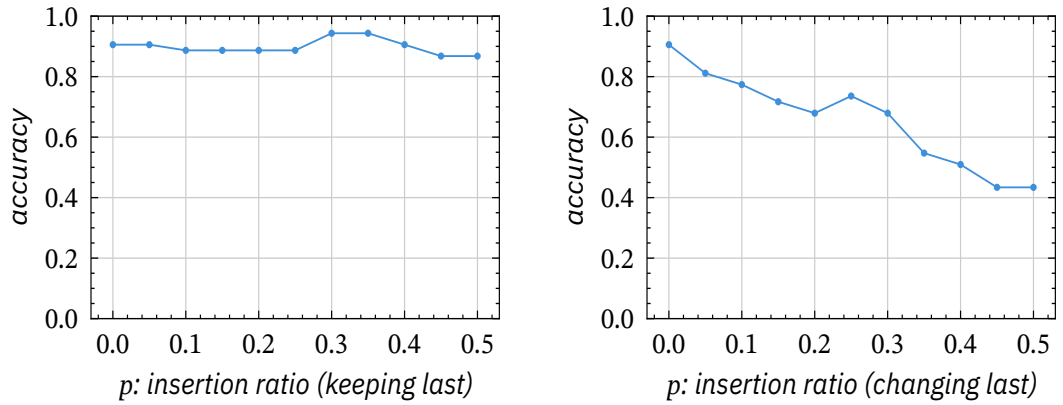


Figure 5.1: *Progression of accuracy over increasing insertion rate p*

This suggests that the model highly depends on the last prefix event being the correct event to predict the next event.

5.2.2 Dropping Random Events

To simulate dropped messages, we can simply drop random events from our prefix traces before prediction. The percentage of dropped messages is controlled by a parameter p. We validate the prediction outputs again by appending the next step to the *original*, pre-drop prefix and checking for correctness.

For the dropping of random events we explicitly do not consider the case of dropping the last event, since the model would then possibly predict the last event itself. Due to our validation mechanism this would duplicate the last token, immediately treating the model output as incorrect, even though the model output on the given sequence would be perfectly correct.

By dropping p parts of the run, except the last event, the accuracy only reduces slowly compared to the scenario in Figure 5.1. Surprisingly, even when dropping 50% of the events, the accuracy remains at ~**81%** from the previous ~91%. The plot of accuracy can be seen in Figure 5.2.

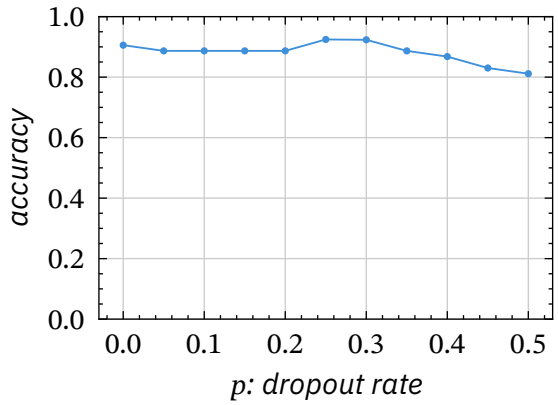


Figure 5.2: *Progression of accuracy over increasing dropout rate p*

The reduced rate of accuracy rate points towards good generalization capabilities of the model when dealing with message drops.

5.2.3 Extended Input Run

The training data just contains traces of the APS, for which one workpiece was processed at a time, from entering the factory at the DPS, up to failing at the AIQS or being dropped off for delivery at the DPS again. We now predict on a singular additional recorded trace, for which nine workpieces are passed into the factory directly after each other, with the orders starting to be processed directly. It consists of 180 steps in total, averaging 20 steps per workpiece.

This run has certain properties our training runs do not have, that can have an impact on the prediction quality:

1. The run contains workpieces of multiple colors. Remember that we use special *meta* color tokens to let the model know what kind of workpiece is processed in this sequence (see Section 4.3). Usually, this token is automatically prepended based on the trace metadata containing the color. Thus, our automatic preprocessing cannot handle the color changes in between the workpiece processes.

2. The run contains previously unseen parallelism. Especially the started processing of the first workpiece, while pieces are picked up from the DPS and passed to the HBW, lets the AGV continuously transfer between DPS, HBW and the processing workstation.

3. The model outputs `<EOS>` tokens when it reaches what it learned to be the end of a sequence and thus a run. In the training data, every run ends after processing one piece. Here, the `<EOS>` should only be output after the processing of multiple workpieces. This type of generalization cannot be expected from our model.

Due to the set of training data sequences never containing such parallel scenarios, there is no proper workaround for the second and third issues. One could retrain a new model on training data containing such runs or split the traces into multiple separate traces and thus create scenarios known to our model. Splitting the traces by workpiece order could create traces from the perspective of individual workpieces, reducing the parallelism and ending individual sequences properly, which our model is capable of handling - we do not evaluate this in this thesis, as our preprocessing is not able to distinct different orders. The coloring issue could be resolved at the time by adding additional color meta-tokens by hand into the token sequence, which would, however, also require changes to our processing specific to this sample.

The APS trace is processed following the same procedures for training and validation data as seen in Section 4.1. We then perform the same evaluation baseline evaluations as before.

For simple next step prediction, our model has an accuracy of **45.25%**. Looking at the incorrect predictions, ~10% incorrectly assumed <EOS> matching our expectation from 3. An additional ~25% the model failed to predict additional AGV movements never seen in that context in the training data, and ~33% originate from the repeated picks and drops from the DPS and the HBW, stemming from the insertion of additional workpieces into the factory. The remaining failures can be mostly attributed to unknown process configurations from missing color information, as the model incorrectly predicts the sequences of process modules.

Thus, the accuracy, while seeming low, matches the expectations due to the limits explained above. The hypothesis is supported by the accuracy of the top 2 prediction of just **53.63%**, as many of the correct steps of the large trace are not even considered a valid option by the model.

5.3 Model Performance

Predictive process monitoring is mostly done for ongoing processes (Di Francescomarino et al., 2026), and possible in an completely online setting, predicting on data just as it arrives at the system. Latter scenario is especially relevant, if immediate feedback for ongoing processes or future processes is required. Both the *resource requirements* and the *amount of time* required for singular predictions are important factors to consider when discussing online deployment.

Resource requirements are a non-issue for our model. The prediction itself consumed at most 400MB of memory during longer-running benchmarks, with up to 60% single CPU usage and 5% GPU usage. Not using 100% of the CPU can be explained by offloaded work to the GPU, during which the CPU is not needed by the program. Our model only consists of *42466* floating-point parameters, thus VRAM with a fully loaded model usage is limited as well.

The individual prediction times amounted to an average of 40ms including reloading the model after each prediction. The first prediction requires loading the model from the filesystem, resulting in ~400ms loading times, later predictions only reloading the model from mapped memory then average at only 34ms. These times can certainly be enhanced by not reloading the model after every prediction, however the current infrastructure does not provide the functionality for consistently keeping the model loaded.

Combining these two results, we can infer that online usage is certainly possible with this model, even though it would require some infrastructure build-up to

support live translation of the MQTT events into the steps and pipelining that into the model.

6

Conclusion and Future Work

6.1 Conclusion

This thesis presents an approach to using the transformer deep learning architecture to perform predictive process monitoring, especially next-event prediction, in concurrent systems. This approach starts by modeling relevant events as Heraklit *steps* (Fettke & Reisig, 2021), which are then interpreted as *tokens* for a modified transformer architecture by Vaswani et al. (2017).

To evaluate the approach, we introduce a *correctness* measure of a prediction based on prefix runs. We rely on the Heraklit composition calculus to allow arbitrary ordering of causally unrelated events, while enforcing the order of causally related events.

Our approach is then evaluated on the Fischertechnik *Agile Production Simulation* to perform next-event prediction within the distributed production system from the view of the central control unit. We train our model on a limited set of event traces of singular workpiece processing throughout the factory.

The resulting model shows a high accuracy of **91.07%** for single next step prediction, and when considering the two possible next steps with the highest probabilities, our model reaches **100%** accuracy on the validation data set.

We also perform an analysis of the generalization performance of the model on special scenarios of the factory. As our factory communicates via MQTT over a network, we simulate scenarios with dropped messages and random arrival of late, unrelated messages. With dropped messages, the model accuracy only falls to ~81% with 50% of the messages being dropped. Similarly, with random unrelated messages, the performance of our model only decreases slightly to 86% when the latest event stays the same, but falls down to ~50% when new events are also inserted after the last original event.

This is expected, as our model is prepared for generalization due to the use of *cross validation* and *dropout* during the training, however it highly relies on the last step to predict the next.

Our approach shows clear success on the Fischertechnik APS, while creating a model performant enough to be run on conventional mobile devices and prediction times that would allow use in an online production environment.

To conclude, our approach creates a model that properly predicts next events, while allowing the reordering of non-causally related events. Thus, an application for parallel or concurrent systems such as a distributed production environment is reasonable.

6.2 Limitations and Future Work

In the following section we want to highlight certain limitations and present relevant open problems for future work.

Our case study relies on a dataset of only 10 runs through the system. While the resulting performance from such a small dataset is impressive, the training data obviously did not catch all edge cases for the model to learn. More data can be extracted from the APS, and more complex scenarios can be reflected in the training data, such as parallel processing of multiple workpieces or multiple AGVs running in the same system, as highlighted in Section 5.2.3.

The dataset also did not contain the metadata from the raw MQTT messages, such as the `DUP` flag to identify resends of messages due to the QoS levels. This additional protocol information could make future data pre-processing more reliable.

We did not focus on extracting the parameters for specific actions, such as the amount of time the MILL should perform its action. Adding these type of parameters to the modeling and encoding could open up the prediction of more fine-grained prediction within the APS. This is a clear limitation on what kind of processes can be modeled by the infrastructure at the moment, as parameters need to be included into the tokens directly. Adding byte-pair encoding instead of doing a 1-1 step to token mapping could simplify this in future work.

We perform no comparison to other prediction model architectures, as this would be out of scope for this thesis. To clearly isolate the performance benefits coming from the transformer architecture, it remains to benchmark other, possibly simpler traditional machine learning models. Especially with the fundamental simplicity of the factory, traditional machine learning models could already perform quite well for limited scenarios.

Some further actions on the basis of our model could be to

- Extend our discrete pipeline into one continuous live algorithm, that can consume the live logs as an input and output concrete predictions as an output. This should be straightforward, as the singular steps of the pipeline already exist; however the current pipeline would need to be adjusted to combine all scripts into one module.
- Focus on explainability of the model. During generalization testing, we realized that especially the last token has a big influence on the next predictions. An analysis to identify which parts of the sequence the model puts its *attention* to would increase explainability, and thus trust in the model.
- Extend the Fischertechnik APS software to create early indicators for failure during the process, such as milling longer or by providing simulated tool wear. This would ensure that a process prediction on the APS can base failure or success predictions on run information.

Bibliography

Aalst, W. van der. (2016). Data Science in Action. In *Process Mining: Data Science in Action* (2nd ed.). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-49851-4_1

Bukhsh, Z. A., Saeed, A., & Dijkman, R. M. (2021). ProcessTransformer: Predictive business process monitoring with Transformer Network. *Corr*. https://arxiv.org/abs/2104.00721

Camargo, M., Dumas, M., & González-Rojas, O. (2019). Learning accurate LSTM models of business processes. *International Conference on Business Process Management*, 286–302. https://doi.org/10.1007/978-3-030-26619-6_19

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186.

Di Francescomarino, C., Donadello, I., & Maggi, F. M. (2026). *Predictive Process Monitoring*. Springer. https://doi.org/10.1007/978-3-031-08848-3_10

Evermann, J., Rehse, J.-R., & Fettke, P. (2017). Predicting process behaviour using deep learning. *Decision Support Systems*, *100*, 129–140. https://doi.org/10.1016/j.dss.2017.04.003

Fettke, P., & Reisig, W. (2021). Modelling service-oriented systems and cloud services with Heraklit. *Advances in Service-Oriented and Cloud Computing*, 77–89. https://doi.org/10.1007/978-3-030-71906-7_7

Fettke, P., & Reisig, W. (2022). Modularization, composition, and hierarchization of Petri nets with Heraklit. *Corr*. https://arxiv.org/abs/2202.01830

Fischertechnik. (n.d.). *Fischertechnik Agile Production Simulation*. Retrieved June 11, 2026, from https://www.fischertechnik.de/de-de/industrie-und-hochschulen/technische-dokumente/simulieren/agile-production-simulation#uebersicht

Han, K., Xiao, A., Wu, E., Guo, J., XU, C., & Wang, Y. (2021). Transformer in Transformer. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, & J. W. Vaughan (Eds.), *Advances in Neural Information Processing Systems: Vol. 34. Advances in Neural Information Processing Systems*. https://proceedings.neurips.cc/

paper_files/paper/2021/file/854d9fca60b4bd07f9bb215d59ef5561-Paper.pdf

Kingma, D. P., & Ba, J. (2017). Adam: A method for stochastic optimization. In *International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980

Kulkarni, A., Shivananda, A., Kulkarni, A., & Gudivada, D. (2023). The ChatGPT architecture: An in-depth exploration of OpenAI's conversational language model. In *Applied Generative AI for Beginners: Practical Knowledge on Diffusion Models, ChatGPT, and Other LLMs* (pp. 55–77). Apress. https://doi.org/10.1007/978-1-4842-9994-4_4

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., & Stoica, I. (2023). Efficient memory management for Large Language Model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626. https://doi.org/10.1145/3600006.3613165

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations*. https://arxiv.org/abs/1711.05101

Mehdiyev, N., Evermann, J., & Fettke, P. (2020). A novel business process prediction model using a Deep Learning method. *Business & Information Systems Engineering*, *62*(2), 143–157. https://doi.org/10.1007/s12599-018-0551-3

Muehlen, M. z., & Ho, D. T.-Y. (2006). Risk Management in the BPM Lifecycle. In C. J. Bussler & A. Haller (Eds.), *Business Process Management Workshops: Business Process Management Workshops*. https://doi.org/10.1007/11678564_42

Neu, D. A., Lahann, J., & Fettke, P. (2022). A systematic literature review on state-of-the-art deep learning methods for process prediction. *Artificial Intelligence Review*, *55*(2), 801–827. https://doi.org/10.1007/s10462-021-09960-8

Ni, W., Zhao, G., Liu, T., Zeng, Q., & Xu, X. (2023). Predictive business process monitoring approach based on hierarchical Transformer. *Electronics*, *12*(6). https://doi.org/10.3390/electronics12061273

Pfeiffer, P., Abb, L., Fettke, P., & Rehse, J.-R. (2025). Learning from the data to predict the process. *Business & Information Systems Engineering*, *67*(3), 357–383. https://doi.org/10.1007/s12599-025-00936-4

Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., & others. (2018). Improving language understanding by generative pre-training. *Openai Blog*.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., & others. (2019). Language models are unsupervised multitask learners. *Openai Blog*.

Rebmann, A., Schmidt, F. D., Glavaš, G., & Aa, H. van der. (2025). On the potential of large language models to solve semantics-aware process mining tasks. *Process Science*, *2*(1), 10. https://doi.org/10.1007/s44311-025-00019-3

Tax, N., Verenich, I., La Rosa, M., & Dumas, M. (2017). Predictive business process monitoring with LSTM Neural Networks. In E. Dubois & K. Pohl (Eds.), *Advanced Information Systems Engineering: Advanced Information Systems Engineering*. https://doi.org/10.1007/978-3-319-59536-8_30

Toldinas, J., Lozinskis, B., Baranauskas, E., & Dobrovolskis, A. (2019). MQTT Quality of Service versus energy consumption. *2019 23rd International Conference Electronics*, , 1–4. https://doi.org/10.1109/ELECTRONICS.2019.8765692

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

Yassein, M. B., Shatnawi, M. Q., Aljwarneh, S., & Al-Hatmi, R. (2017). Internet of Things: Survey and open issues of MQTT protocol. *2017 International Conference on Engineering & MIS (ICEMIS)*, , 1–6. https://doi.org/10.1109/ICEMIS.2017.8273112

Zhu, T. (2020). Analysis on the applicability of the Random Forest. *Journal of Physics: Conference Series*, *1607*(1), 12123. https://doi.org/10.1088/1742-6596/1607/1/012123